Analisis Perbandingan Performa Framework Frontend VueJS dan Svelte pada Prototipe Website Listing Properti Berbasis Studi Kasus Airbnb Berdasarkan Segi Komputasional dan Pengalaman Pengguna

LAPORAN SKRIPSI

Untuk memenuhi sebagian persyaratan
memperoleh gelar Sarjana Komputer

Disusun oleh:

Exel Boy Alfanso
NIM: 235150201111019



PROGRAM STUDI TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
MALANG
2026

DAFTAR ISI

BAB 1 PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 3
1.3 Tujuan 4
1.4 Manfaat 4
1.5 Batasan Masalah 4
1.6 Sistematika Pembahasan 5

BAB 2 LANDASAN KEPUSTAKAAN 6
2.1 Tinjauan Penelitian Terdahulu 6
2.2 Framework Frontend 8
2.2.1 Vue.js 8
2.2.2 Svelte 9
2.2.3 Dasar Perbandingan Vue.js dan Svelte 9
2.3 Website Listing Properti sebagai Workload Pengujian 10
2.4 Metrik Performa Google Lighthouse 10
2.4.1 First Contentful Paint 11
2.4.2 Largest Contentful Paint 11
2.4.3 Speed Index 12
2.4.4 Total Blocking Time 12
2.4.5 Cumulative Layout Shift 12
2.5 Efisiensi Sumber Daya 12
2.6 Validasi Kesetaraan Prototipe 13
2.7 User Experience dan Usability 13
2.7.1 User Experience Questionnaire 14
2.7.2 Think Aloud Protocol 16
2.8 Statistika Deskriptif dan Inferensial 17
2.8.1 Ukuran Pemusatan dan Penyebaran 17
2.8.2 Analisis Pengukuran Berpasangan 17
2.8.3 Pemilihan Uji dan Pelaporan 17
2.9 Kerangka Konseptual Penelitian 18

BAB 3 METODOLOGI PENELITIAN 19
3.1 Rancangan Penelitian 19
3.2 Persiapan dan Pengembangan Prototipe 20
3.2.1 Arsitektur Pengembangan Prototipe 21
3.2.2 Struktur Tabel Basis Data 22

3.2.3 Logika Backend dan Pseudocode 25
3.2.4 Rancangan UI/UX Prototipe 26
3.2.5 Spesifikasi Fitur dan Pseudocode 27
3.2.6 Perbandingan Implementasi VueJS dan Svelte 29
3.2.7 Versi Pustaka dan Perangkat Pengembangan 30
3.3 Validasi Kesetaraan Fitur 31
3.3.1 Unit Testing 31
3.3.2 Integration Testing 32
3.3.3 End-to-End Testing 32
3.4 Pengujian Performa Komputasional 33
3.4.1 Lingkungan Pengujian 33
3.4.2 Pengujian Metrik Performa Google Lighthouse 34
3.4.3 Pengujian Efisiensi Sumber Daya 35
3.5 Evaluasi User Experience 35
3.5.1 Partisipan Penelitian 35
3.5.2 Skenario Pengujian 36
3.5.3 Prosedur Pengujian 37
3.5.4 User Experience Questionnaire 38
3.5.5 Think Aloud Protocol 39
3.6 Teknik Analisis Data 40
3.6.1 Analisis Performa Komputasional 40
3.6.2 Analisis UEQ 43
3.6.3 Analisis Think Aloud Protocol 43
3.7 Analisis Komparatif Hasil Pengujian 44
3.8 Jadwal Penelitian 45

BAB 4 HASIL **46**
4.1 Hasil Pengujian Performa Komputasional 46
4.1.1 Cakupan Data Pengujian 46
4.1.2 Statistika Deskriptif Metrik Lighthouse 46
4.1.3 Statistika Inferensial Metrik Lighthouse 48
4.1.4 Statistika Deskriptif Efisiensi Sumber Daya 49

DAFTAR REFERENSI **51**

DAFTAR TABEL

2.1 Perbandingan Penelitian Terdahulu 7
2.2 Batas Interpretasi Metrik Performa Lighthouse 12

3.1 Variabel Kontrol Penelitian 21
3.2 Struktur Tabel Referensi Basis Data Prototipe 23
3.3 Struktur Tabel Properti dan Relasi Basis Data Prototipe 23
3.4 Struktur Antarmuka dan Fungsi UX Prototipe 27
3.5 Fitur Prototipe dan Perilaku yang Divalidasi 28
3.6 Padanan Konstruksi Implementasi VueJS dan Svelte 29
3.7 Versi Pustaka dan Perangkat Frontend VueJS dan Svelte 31
3.8 Skenario Pengujian Kesetaraan Fitur 32
3.9 Konfigurasi Lingkungan Pengujian 33
3.10 Strata Perangkat Evaluasi User Experience 36
3.11 Butir User Experience Questionnaire Versi Bahasa Indonesia 38
3.12 Pemilihan Uji Komparatif Metrik Performa 41
3.13 Threshold Metrik Performa Lighthouse 42
3.14 Instrumen dan Alat Pengukuran 43

4.1 Statistika Deskriptif Metrik Lighthouse: Pemusatan dan Penyebaran 46
4.2 Statistika Deskriptif Metrik Lighthouse: Kuartil dan Rentang 47
4.3 Selisih Deskriptif Metrik Lighthouse (Vue.js–Svelte) 47
4.4 Kategori Lighthouse Berdasarkan Nilai Maksimum Observasi 48
4.5 Normalitas Shapiro–Wilk Selisih Pasangan 48
4.6 Hasil Inferensial Berpasangan Metrik Lighthouse 49
4.7 Statistika Deskriptif Efisiensi Sumber Daya: Pemusatan dan Penyebaran 49
4.8 Statistika Deskriptif Efisiensi Sumber Daya: Kuartil dan Rentang 50
4.9 Selisih Deskriptif Efisiensi Sumber Daya (Vue.js–Svelte) 50

DAFTAR GAMBAR

2.1 Alur Sederhana Mekanisme Rendering Vue.js 9
2.2 Alur Sederhana Mekanisme Rendering Svelte 9
2.3 Contoh Tampilan Laporan Audit Google Lighthouse 11
2.4 Ringkasan Skala dan Aspek yang Diukur oleh UEQ 14
2.5 Struktur Enam Skala pada UEQ 15
2.6 Alur Evaluasi UEQ 16
2.7 Kerangka Konseptual Perbandingan Vue.js dan Svelte 18

3.1 Diagram Alir Tahapan Penelitian 20
3.2 Arsitektur Aplikasi dan Runtime Backend Prototipe 21
3.3 ERD Tabel Katalog Properti 25
3.4 Timeline Pengerjaan Skripsi 45

4.1 Perbandingan Mean dan SD FCP, LCP, serta Speed Index 47
4.2 Q–Q Plot Selisih Pasangan FCP, LCP, dan Speed Index 48
4.3 Mean Selisih Pasangan dan CI Bootstrap 95% 49
4.4 Perbandingan Mean dan SD Ukuran Transfer 50

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Perkembangan aplikasi web modern menuntut website untuk memiliki performa yang cepat, responsif, dan efisien dalam penggunaan sumber daya. Kebutuhan tersebut menjadi semakin penting pada website berbasis listing yang menampilkan banyak konten visual dan elemen antarmuka secara bersamaan. Pada website listing properti, misalnya, halaman dapat memuat gambar properti, informasi lokasi, harga, rating, serta berbagai komponen interaktif seperti pencarian, penyaringan, pengurutan, dan pemuatan data tambahan. Karakteristik tersebut menyebabkan sisi frontend memiliki peran penting terhadap waktu pemuatan halaman, respons terhadap interaksi pengguna, stabilitas tampilan, serta jumlah sumber daya yang perlu diproses oleh browser.

Skala aktivitas pada platform akomodasi digital dapat dilihat dari laporan Airbnb. Dalam *Q4 2025 Shareholder Letter*, Airbnb (2026, hlm. 2 dan 23) melaporkan 533 juta *Nights and Seats Booked* sepanjang tahun 2025, meningkat 8% dibandingkan tahun sebelumnya. Secara kuartalan, jumlah tersebut terdiri atas 143,1 juta pada Q1, 134,4 juta pada Q2, 133,6 juta pada Q3, dan 121,9 juta pada Q4. Metrik tersebut merupakan jumlah malam akomodasi dan kursi untuk layanan serta pengalaman yang dipesan, setelah memperhitungkan pembatalan dan perubahan pada periode pelaporan. Angka ini menggambarkan volume aktivitas pemesanan, sehingga tidak dapat disamakan dengan jumlah pengguna unik maupun kunjungan situs. Skala aktivitas tersebut memperkuat relevansi Airbnb sebagai konteks studi kasus platform listing properti dalam penelitian ini.

Dukungan terhadap pemilihan skenario interaksi tersebut dapat ditemukan pada pola penggunaan Airbnb. (Deng et al., 2023) menjelaskan bahwa sekitar 90% pengunjung memiliki lokasi tujuan tertentu dan menggunakan kotak pencarian dengan memasukkan lokasi, tanggal, dan jumlah tamu. Jumlah tamu selanjutnya menyaring listing berdasarkan kapasitas. Hasil pencarian disajikan sebagai daftar yang telah diberi peringkat, dan sebagian besar permintaan menghasilkan lebih dari satu halaman kartu listing yang dapat terus ditinjau dari permintaan pencarian yang sama. Seorang pengguna rata-rata juga dapat menghasilkan 10 hingga 20 permintaan melalui kotak pencarian selama periode kunjungan dua hingga tiga minggu. Temuan tersebut menunjukkan bahwa pencarian, penyaringan, perubahan urutan hasil, dan penelusuran data lintas halaman merupakan aktivitas yang relevan dalam alur eksplorasi akomodasi. Pada prototipe penelitian ini, proses tersebut dioperasionalkan sebagai pencarian, *advanced filter*, *sorting*, dan *load more* untuk membentuk skenario interaksi yang setara pada VueJS dan Svelte. *Sorting* manual dan *load more* digunakan sebagai representasi terkendali dari perubahan peringkat dan penelusuran hasil multipage, bukan sebagai replikasi persis kontrol antarmuka Airbnb.

VueJS dan Svelte merupakan framework frontend berbasis komponen yang dapat digunakan untuk membangun antarmuka web interaktif. Meskipun memiliki tujuan yang serupa, keduanya menggunakan pendekatan berbeda dalam proses

pembaruan antarmuka. VueJS menggunakan sistem reaktivitas yang bekerja bersama mekanisme Virtual DOM pada saat aplikasi berjalan, sedangkan Svelte menggunakan pendekatan berbasis compiler dengan mengompilasi komponen menjadi kode JavaScript pada proses build. Perbedaan pendekatan tersebut berpotensi menghasilkan karakteristik performa yang berbeda, baik dari sisi proses rendering, jumlah JavaScript yang dikirimkan ke browser, penggunaan sumber daya, maupun respons aplikasi ketika terjadi perubahan data.

Penelitian sebelumnya telah menunjukkan bahwa perbedaan strategi rendering pada framework frontend dapat menghasilkan karakteristik performa yang berbeda. (Ollila, Mäkitalo and Mikkonen, 2022) membandingkan strategi rendering Angular, React, Vue, Svelte, dan Blazor menggunakan sejumlah benchmark untuk mengukur perubahan biaya rendering seiring meningkatnya kompleksitas aplikasi. Penelitian tersebut menunjukkan adanya perbedaan pola performa berdasarkan strategi rendering yang digunakan oleh masing-masing framework. Selain itu, (Putra et al., 2025) membandingkan Svelte, React, dan Vue menggunakan aplikasi web sederhana yang dibuat secara identik dengan pengukuran berupa rendering time, bundle size, penggunaan memori saat idle, dan efisiensi operasi DOM. Penelitian tersebut menunjukkan bahwa perbedaan arsitektur framework dapat memengaruhi performa teknis aplikasi.

(Fahreza, Kharisma and Marji, 2025) membandingkan performa rendering Next.js dan SvelteKit pada aplikasi e-commerce menggunakan metrik TBT, CLS, dan Speed Index untuk metode CSR, SSR, dan SSG. Hasil pengukuran berulang dirangkum menggunakan mean dan simpangan baku, kemudian dianalisis melalui uji normalitas serta pengujian hipotesis parametrik atau nonparametrik. Penelitian tersebut menunjukkan bahwa analisis statistik dapat digunakan untuk membedakan selisih hasil pengukuran dari variasi antar-run. Meskipun objek framework, domain aplikasi, dan metode renderingnya berbeda, alur pengujian berulang dan analisis inferensialnya menjadi salah satu referensi penyusunan metode analisis performa pada penelitian ini.

Meskipun demikian, penelitian terdahulu tersebut masih berfokus pada benchmark rendering atau implementasi aplikasi yang relatif sederhana. Kondisi tersebut belum sepenuhnya menggambarkan karakteristik aplikasi web yang memiliki banyak konten visual serta melibatkan beberapa bentuk interaksi dan perubahan data dalam satu aplikasi. Oleh karena itu, masih diperlukan kajian mengenai bagaimana perbedaan pendekatan VueJS dan Svelte menghasilkan karakteristik performa ketika diterapkan pada prototipe aplikasi yang memiliki workload lebih representatif, seperti website listing properti yang mencakup pemuatan sejumlah gambar dan komponen berulang, pencarian, penyaringan, pengurutan, pemuatan data tambahan, dan navigasi menuju detail properti. Selain itu, evaluasi yang menggabungkan metrik kecepatan pemuatan, progres visual, waktu pemblokiran, stabilitas tampilan, dan efisiensi sumber daya dapat memberikan gambaran yang lebih menyeluruh mengenai perbedaan performa kedua framework.

Metrik performa tersebut belum cukup untuk menyimpulkan pengalaman pengguna secara keseluruhan. Wehner et al. (2023) membandingkan pengukuran objektif menggunakan Google Lighthouse dengan penilaian subjektif *web Quality*

of Experience dan menemukan bahwa metrik Core Web Vitals pada konteks studi mereka kurang mampu memprediksi penilaian pengguna daripada yang diharapkan. Temuan tersebut tidak menghilangkan kegunaan metrik teknis, tetapi menunjukkan bahwa waktu tampil konten, pemblokiran *main thread*, dan kestabilan visual tidak secara langsung menjelaskan apakah pengguna memahami kontrol, dapat menyelesaikan alur, atau merasa interaksi efisien dan menyenangkan. Mortazavi et al. (2024) juga menunjukkan bahwa pengalaman pengguna bersifat multidimensi dan mencakup, antara lain, dimensi usability, utility, hedonis, emosi, dan sensoris. Oleh karena itu, penelitian ini melengkapi pengukuran performa komputasional dengan UEQ untuk mengukur persepsi pengguna serta *Think Aloud Protocol* untuk mengidentifikasi hambatan yang terjadi selama pelaksanaan tugas. Ketiga sumber data diperlakukan sebagai bukti yang saling melengkapi, bukan sebagai pengganti satu sama lain.

Berdasarkan kesenjangan tersebut, penelitian ini mengembangkan dua prototipe website listing properti menggunakan VueJS dan Svelte dengan fitur dan perilaku yang dibuat setara. Kedua implementasi menggunakan desain, dataset, aset, struktur konten, dan service backend yang sama. ElysiaJS digunakan sebagai service backend untuk menyediakan data melalui API, sedangkan Eden Treaty digunakan untuk mendukung komunikasi API yang type-safe antara frontend dan backend. Kesetaraan tersebut diterapkan agar perbedaan hasil pengujian dapat lebih difokuskan pada karakteristik implementasi frontend masing-masing framework, bukan akibat perbedaan data, desain, fitur, atau layanan backend.

Prototipe yang dikembangkan mengacu pada karakteristik umum website listing properti modern, khususnya pola penyajian daftar properti yang kaya konten visual serta mendukung fitur seperti pencarian, penyaringan, pengurutan, halaman detail, dan pemuatan data secara bertahap. Airbnb dapat digunakan sebagai salah satu referensi karakteristik antarmuka dan pola interaksi pada domain tersebut. Namun, penelitian ini tidak bertujuan untuk mereplikasi Airbnb secara penuh maupun mengevaluasi teknologi yang digunakan oleh Airbnb, melainkan menggunakan karakteristik website listing properti sebagai konteks untuk melakukan perbandingan performa VueJS dan Svelte.

1.2 Rumusan Masalah

1. Bagaimana perbedaan performa komputasional antara VueJS dan Svelte pada prototipe website listing properti berdasarkan First Contentful Paint (FCP), Largest Contentful Paint (LCP), Speed Index, Total Blocking Time (TBT), Cumulative Layout Shift (CLS), dan efisiensi sumber daya?

2. Bagaimana perbandingan User Experience pada prototipe website listing properti yang dikembangkan menggunakan VueJS dan Svelte berdasarkan hasil User Experience Questionnaire (UEQ)?

3. Permasalahan User Experience apa saja yang ditemukan pada prototipe website listing properti yang dikembangkan menggunakan VueJS dan Svelte berdasarkan hasil *Think Aloud Protocol*?

4. Bagaimana hasil perbandingan performa komputasional dan User Experience antara VueJS dan Svelte dapat menjadi pertimbangan dalam pemilihan framework frontend untuk pengembangan website listing yang kaya konten visual dan memiliki fitur interaktif?

1.3 Tujuan

1. Menganalisis perbedaan performa komputasional antara VueJS dan Svelte pada prototipe website listing properti berdasarkan First Contentful Paint (FCP), Largest Contentful Paint (LCP), Speed Index, Total Blocking Time (TBT), Cumulative Layout Shift (CLS), dan efisiensi sumber daya.
2. Menganalisis perbandingan User Experience pada prototipe website listing properti yang dikembangkan menggunakan VueJS dan Svelte berdasarkan hasil User Experience Questionnaire (UEQ).
3. Mengidentifikasi dan membandingkan permasalahan User Experience pada prototipe website listing properti yang dikembangkan menggunakan VueJS dan Svelte melalui *Think Aloud Protocol*.
4. Menganalisis hasil perbandingan performa komputasional dan User Experience antara VueJS dan Svelte sebagai pertimbangan dalam pemilihan framework frontend untuk pengembangan website listing yang kaya konten visual dan memiliki fitur interaktif.

1.4 Manfaat

Penelitian ini diharapkan memberi manfaat sebagai berikut:

1. Memberikan bukti empiris mengenai karakteristik performa komputasional, efisiensi sumber daya, dan pengalaman pengguna pada implementasi VueJS dan Svelte dalam konteks prototipe website listing properti.
2. Menjadi bahan pertimbangan bagi pengembang dalam memilih VueJS atau Svelte untuk website listing yang kaya konten visual dan memiliki fitur interaktif.
3. Menjadi referensi bagi penelitian selanjutnya mengenai perbandingan framework frontend dengan pengendalian fitur, dataset, aset, dan skenario pengujian yang setara.

1.5 Batasan Masalah

1. Penelitian hanya membandingkan implementasi frontend menggunakan VueJS dan Svelte. ElysiaJS digunakan sebagai service backend yang sama pada kedua prototipe dan tidak menjadi objek utama perbandingan.
2. Prototipe yang dikembangkan dibatasi pada fitur utama website listing properti, yaitu halaman daftar properti, halaman detail properti, pencarian, penyaringan

(filter), pengurutan (sorting), pagination atau load more, serta wishlist sederhana.

3. Airbnb digunakan sebagai referensi karakteristik antarmuka dan pola interaksi website listing properti, bukan sebagai objek yang direplikasi secara penuh maupun sebagai objek evaluasi teknologi yang digunakan oleh Airbnb.

4. Data properti, lokasi, tipe properti, fasilitas, rating, host, dan gambar yang digunakan pada kedua prototipe merupakan data dummy atau aset bebas lisensi yang dibuat sama untuk kedua implementasi.

5. Penelitian tidak mencakup fitur authentication, authorization, pembayaran, transaksi pemesanan sebenarnya, chat, host dashboard, admin dashboard, sistem kalender real-time, dan integrasi peta sebenarnya.

6. Pengujian performa komputasional dilakukan pada production build dengan kondisi perangkat, browser, viewport, dataset, aset, fitur, dan skenario pengujian yang dibuat sama untuk implementasi VueJS dan Svelte.

7. Pengukuran performa komputasional dibatasi pada metrik Google Lighthouse yang terdiri atas First Contentful Paint (FCP), Largest Contentful Paint (LCP), Speed Index, Total Blocking Time (TBT), dan Cumulative Layout Shift (CLS), serta metrik efisiensi sumber daya yang meliputi ukuran JavaScript, total transferred size, dan request count.

8. Evaluasi User Experience dilakukan terhadap kedua prototipe menggunakan skenario penggunaan yang sama. Data kuantitatif diperoleh melalui User Experience Questionnaire (UEQ), sedangkan data kualitatif mengenai pemikiran, ekspektasi, dan kesulitan pengguna diperoleh melalui *Think Aloud Protocol*.

9. Penelitian berfokus pada perbandingan VueJS dan Svelte dalam konteks prototipe website listing properti yang dikembangkan. Hasil penelitian tidak ditujukan untuk menentukan framework terbaik secara umum maupun digeneralisasikan terhadap seluruh jenis aplikasi web.

1.6 Sistematika Pembahasan

Skripsi ini disusun dalam enam bab. Bab 1 menjelaskan latar belakang, rumusan masalah, tujuan, manfaat, dan batasan penelitian. Bab 2 memaparkan landasan kepustakaan yang mendukung penelitian. Bab 3 menjelaskan metodologi penelitian, pengembangan prototipe, pengujian, serta teknik analisis data. Bab 4 menyajikan hasil pengujian dan evaluasi. Bab 5 membahas makna hasil penelitian dalam menjawab rumusan masalah, keterkaitannya dengan teori dan penelitian terdahulu, serta keterbatasan penelitian. Bab 6 memuat kesimpulan dan saran untuk penelitian selanjutnya.

BAB 2 LANDASAN KEPUSTAKAAN

2.1 Tinjauan Penelitian Terdahulu

Penelitian mengenai framework frontend menunjukkan bahwa strategi pembaruan antarmuka dapat menghasilkan karakteristik performa yang berbeda. Ferreira, Borges, dan Valente (2022) menjelaskan bahwa framework frontend JavaScript telah menjadi bagian penting dalam pengembangan antarmuka berbasis komponen, tetapi keputusan adopsinya tidak hanya dipengaruhi performa. Faktor ekosistem, kemudahan pemeliharaan, dokumentasi, dan kesesuaian dengan kebutuhan proyek juga memengaruhi pilihan teknologi. Oleh karena itu, hasil perbandingan framework perlu dibatasi pada konteks aplikasi dan kondisi pengujian yang digunakan.

Ollila, Mäkitalo, dan Mikkonen (2022) membandingkan strategi rendering Angular, React, Vue, Svelte, dan Blazor melalui serangkaian benchmark yang meningkatkan kompleksitas masukan pada proses rendering. Penelitian tersebut menemukan bahwa biaya rendering dapat bertumbuh dengan pola berbeda dan bahwa strategi yang memperkecil masukan pada proses pembaruan, termasuk optimasi saat kompilasi dan model reaktif, dapat menghasilkan keuntungan pada kondisi tertentu. Penelitian tersebut memberikan dasar untuk menguji konsekuensi strategi rendering, tetapi objeknya berupa benchmark terfokus dan bukan aplikasi katalog yang memuat gambar, komunikasi API, serta beberapa alur interaksi pengguna.

Sofi'ie dan Qoiriah (2023) membandingkan React dan Vue pada pengembangan website melalui pengukuran performa dan penggunaan sumber daya. Penelitian tersebut melaporkan keunggulan Vue pada sebagian besar skenario yang diuji. Akan tetapi, Svelte tidak menjadi objek penelitian sehingga hasilnya belum menjawab perbandingan langsung antara Vue.js dan Svelte.

Fahreza, Kharisma, dan Marji (2025) membandingkan Next.js dan SvelteKit pada aplikasi perdagangan elektronik menggunakan pendekatan *client-side rendering*, *server-side rendering*, dan *static site generation*. Metrik yang digunakan mencakup Total Blocking Time (TBT), Cumulative Layout Shift (CLS), dan Speed Index. Hasil pengukuran berulang diringkas menggunakan mean dan simpangan baku, kemudian dianalisis melalui uji normalitas dan uji hipotesis. Penelitian tersebut menjadi rujukan untuk pengukuran berulang, tetapi objek framework, strategi rendering, domain aplikasi, dan desain analisisnya berbeda dari penelitian ini.

Putra et al. (2025) membandingkan Svelte, React, dan Vue melalui implementasi aplikasi yang dibuat serupa. Pengukuran meliputi waktu rendering, ukuran berkas JavaScript, penggunaan memori, dan efisiensi operasi DOM. Penelitian tersebut menunjukkan bahwa perbedaan arsitektur framework dapat berkaitan dengan perbedaan performa teknis. Namun, implementasi yang diuji relatif sederhana dan belum menggabungkan workload katalog kaya gambar, pencarian, penyaringan, pengurutan, pemuatan data lintas halaman, navigasi detail, serta evaluasi pengalaman pengguna.

Wehner et al. (2023) menghubungkan pengukuran objektif Google Lighthouse dengan penilaian subjektif *web Quality of Experience*. Pada konfigurasi studi mereka, Core Web Vitals ditemukan kurang mampu memprediksi penilaian pengalaman pengguna daripada yang diharapkan. Penelitian tersebut paling dekat dengan kebutuhan untuk membedakan hasil pengukuran teknis dan penilaian pengguna, tetapi fokusnya terbatas pada aspek pemuatan halaman dan tidak mengevaluasi usability maupun pengalaman interaksi pada alur tugas aplikasi. Penelitian ini memperluas sudut pandang tersebut dengan mengukur performa laboratorium sekaligus mengevaluasi persepsi dan hambatan pengguna pada alur pencarian, penyaringan, pengurutan, *load more*, navigasi detail, dan wishlist.

Ringkasan posisi penelitian terdahulu terhadap penelitian ini ditunjukkan pada Tabel 2.1.

Tabel 2.1 Perbandingan Penelitian Terdahulu

Penelitian	**Objek dan Pendekatan**	**Aspek Utama**	**Perbedaan dengan Penelitian Ini**
Ollila, Mäkitalo, dan Mikkonen (2022)	Angular, React, Vue, Svelte, dan Blazor melalui benchmark rendering	Pertumbuhan biaya rendering terhadap kompleksitas aplikasi	Belum menggunakan aplikasi katalog kaya gambar dan belum mengevaluasi pengalaman pengguna
Sofi'ie dan Qoiriah (2023)	React dan Vue pada pengembangan website	Performa dan efisiensi sumber daya	Tidak membandingkan Vue.js dengan Svelte
Fahreza, Kharisma, dan Marji (2025)	Next.js dan SvelteKit pada aplikasi perdagangan elektronik	TBT, CLS, Speed Index, dan analisis statistik pengukuran berulang	Framework, strategi rendering, dan domain aplikasi berbeda
Putra et al. (2025)	Svelte, React, dan Vue pada aplikasi sederhana yang dibuat serupa	Waktu rendering, ukuran JavaScript, memori, dan operasi DOM	Workload belum mencakup seluruh alur katalog properti dan evaluasi pengalaman pengguna
Wehner et al. (2023)	Pengukuran Lighthouse dan studi subjektif *web Quality of Experience*	Hubungan Core Web Vitals dengan pengalaman pemuatan yang dirasakan	Tidak membandingkan framework dan tidak mengevaluasi usability pada alur tugas aplikasi

Sumber: Diolah penulis berdasarkan Ollila, Mäkitalo, dan Mikkonen (2022), Sofi'ie dan Qoiriah (2023), Fahreza, Kharisma, dan Marji (2025), Putra et al. (2025), serta Wehner et al. (2023).

Berdasarkan penelitian terdahulu tersebut, terdapat ruang penelitian untuk membandingkan Vue.js dan Svelte pada prototipe yang memiliki workload lebih representatif daripada benchmark terisolasi. Penelitian ini mengisi ruang terse-

but melalui dua implementasi frontend yang memiliki desain, fitur, dataset, aset, kontrak API, dan skenario penggunaan yang setara. Perbandingan mencakup performa pemuatan, efisiensi sumber daya, dan pengalaman pengguna. React hanya muncul sebagai objek pada beberapa penelitian terdahulu dan tidak menjadi perlakuan dalam eksperimen ini.

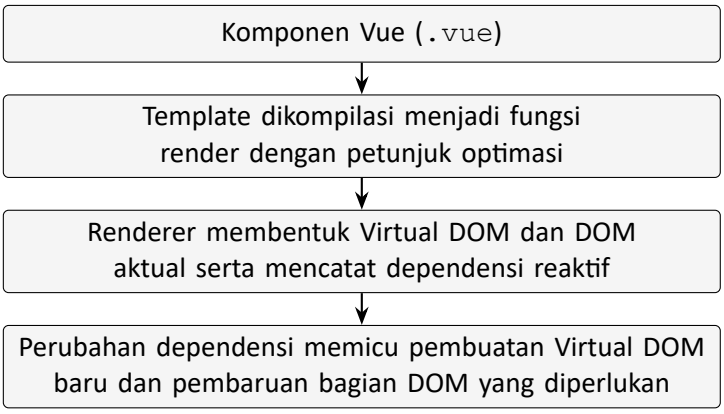
2.2 Framework Frontend

Framework frontend merupakan kerangka kerja untuk membangun antarmuka aplikasi web melalui komponen, pengelolaan status, dan mekanisme pembaruan tampilan. Framework memberi model deklaratif agar pengembang menyatakan tampilan yang diinginkan berdasarkan data, sedangkan framework menerjemahkan perubahan data tersebut menjadi perubahan pada Document Object Model (DOM). Perbedaan cara menerjemahkan perubahan status menjadi pembaruan DOM menjadi salah satu dasar perbandingan Vue.js dan Svelte (Ollila, Mäkitalo, dan Mikkonen, 2022).

2.2.1 Vue.js

Vue.js merupakan framework frontend berbasis komponen. Dokumentasi Vue.js menjelaskan bahwa template dikompilasi menjadi fungsi render yang menghasilkan Virtual DOM. Saat komponen dipasang, renderer membentuk DOM aktual dan mencatat dependensi reaktif yang digunakan. Ketika dependensi berubah, fungsi render dijalankan kembali, representasi Virtual DOM baru dibandingkan dengan representasi sebelumnya, dan perubahan yang diperlukan diterapkan pada DOM aktual (Vue.js, n.d.).

Vue.js tidak hanya mengandalkan perbandingan Virtual DOM pada saat aplikasi berjalan. Kompiler template melakukan analisis statis dan memberikan petunjuk kepada renderer, misalnya melalui penyimpanan node statis dan *patch flags*. Pendekatan ini disebut *compiler-informed Virtual DOM* karena optimasi pada tahap kompilasi membantu mengurangi pekerjaan yang perlu dilakukan renderer pada tahap eksekusi (Vue.js, n.d.). Alur konseptual tersebut ditunjukkan pada Gambar 2.1.



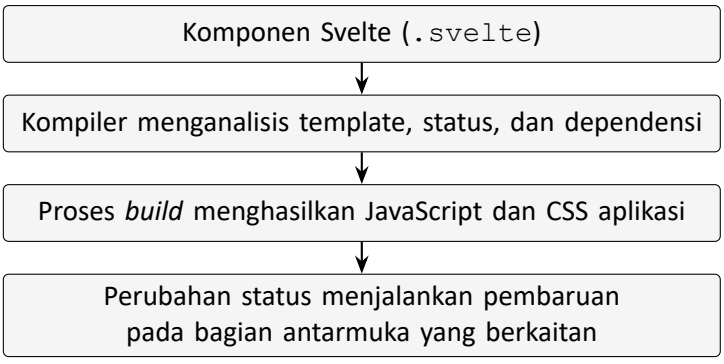
Gambar 2.1 Alur Sederhana Mekanisme Rendering Vue.js

Sumber: Diadaptasi dari Vue.js (n.d.).

2.2.2 Svelte

Svelte merupakan framework antarmuka web yang menggunakan kompiler untuk mengubah komponen deklaratif yang ditulis dengan HTML, CSS, dan JavaScript menjadi kode JavaScript yang teroptimasi (Svelte, n.d.). Pada Svelte 5, status reaktif dapat dinyatakan melalui *runes*. Kompiler menganalisis hubungan antara template dan status sehingga kode hasil kompilasi dapat menjalankan pembaruan ketika dependensi yang berkaitan berubah.

Pendekatan berbasis kompiler tidak berarti bahwa aplikasi Svelte sama sekali tidak memiliki kode runtime. Kode hasil kompilasi tetap menggunakan dukungan runtime yang diperlukan untuk mengelola siklus hidup dan reaktivitas. Perbedaannya adalah sebagian pekerjaan analisis dilakukan pada tahap kompilasi sehingga pembaruan dapat diarahkan pada bagian antarmuka yang berkaitan. Alur konseptual tersebut ditunjukkan pada Gambar 2.2.



Gambar 2.2 Alur Sederhana Mekanisme Rendering Svelte

Sumber: Diadaptasi dari Svelte (n.d.).

2.2.3 Dasar Perbandingan Vue.js dan Svelte

Vue.js dan Svelte sama-sama menggunakan komponen deklaratif serta melakukan optimasi pada tahap kompilasi. Perbedaannya tidak tepat diseder-

hanakan menjadi “runtime melawan tanpa runtime”. Vue.js menggabungkan kompiler template, sistem reaktivitas, dan renderer Virtual DOM yang memperoleh petunjuk dari kompiler. Svelte mengompilasi komponen menjadi kode pembaruan yang diarahkan oleh dependensi reaktif dan tetap menyertakan dukungan runtime yang diperlukan. Perbedaan strategi tersebut dapat memengaruhi jumlah JavaScript, biaya inisialisasi, dan pekerjaan pembaruan antarmuka, tetapi arah serta besar pengaruhnya bergantung pada workload aplikasi (Vue.js, n.d.; Svelte, n.d.; Ollila, Mäkitalo, dan Mikkonen, 2022).

Penelitian ini tidak mengasumsikan salah satu framework selalu lebih cepat. Kedua framework diperlakukan sebagai variabel bebas yang dibandingkan melalui implementasi framework-native dengan perilaku observabel yang setara. Hasil hanya ditafsirkan untuk versi aplikasi, fitur, data, skenario, dan lingkungan pengujian yang ditetapkan.

2.3 Website Listing Properti sebagai Workload Pengujian

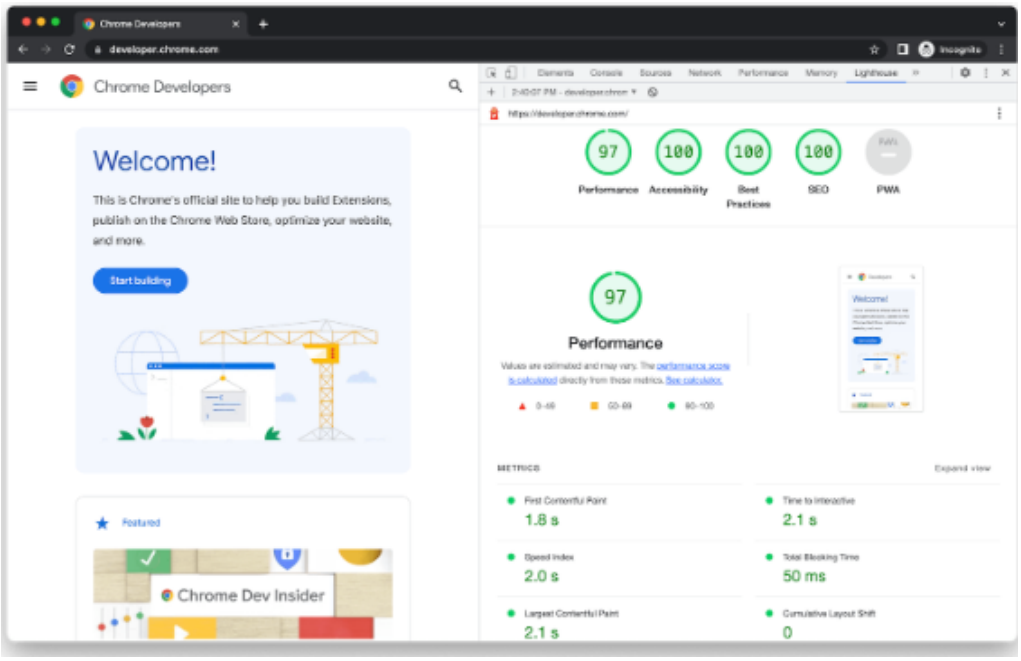
Website listing properti menyajikan katalog akomodasi yang memadukan gambar, lokasi, harga, rating, kapasitas, dan kontrol interaktif. Deng et al. (2023) menjelaskan bahwa sebagian besar pengunjung Airbnb memiliki tujuan tertentu dan menggunakan pencarian dengan informasi lokasi, tanggal, atau jumlah tamu. Hasil pencarian dapat memuat lebih dari satu halaman listing dan pengguna dapat menjalankan beberapa permintaan selama proses eksplorasi. Karakteristik tersebut menunjukkan bahwa katalog properti melibatkan pemuatan konten visual sekaligus perubahan data akibat interaksi.

Pada penelitian ini, karakteristik tersebut dioperasionalkan menjadi halaman daftar dan detail properti, pencarian, penyaringan, pengurutan, *load more*, serta wishlist lokal. Workload ini dipilih karena mencakup pemuatan awal, rendering komponen berulang, pembaruan daftar, komunikasi dengan API, dan navigasi. Airbnb digunakan sebagai referensi pola interaksi, bukan sebagai objek yang direplikasi secara penuh atau sebagai sumber informasi mengenai teknologi internal yang digunakan perusahaan tersebut.

2.4 Metrik Performa Google Lighthouse

Google Lighthouse merupakan alat audit otomatis untuk mengevaluasi kualitas halaman web pada lingkungan laboratorium. Pengujian laboratorium menjalankan halaman pada perangkat, peramban, dan konfigurasi jaringan yang ditentukan sehingga pengukuran dapat diulang. Hasil laboratorium tidak sama dengan data lapangan dari populasi pengguna nyata; karena itu, hasil penelitian ini dibatasi pada lingkungan eksperimen yang dikendalikan.

Contoh tampilan laporan audit Lighthouse ditunjukkan pada Gambar 2.3. Gambar tersebut hanya mengilustrasikan antarmuka dan bukan hasil pengujian prototipe penelitian ini.



Gambar 2.3 Contoh Tampilan Laporan Audit Google Lighthouse

Sumber: Google Chrome Developers (tangkapan layar).

Penelitian menggunakan First Contentful Paint (FCP), Largest Contentful Paint (LCP), Speed Index, Total Blocking Time (TBT), dan Cumulative Layout Shift (CLS). Kelima metrik tersebut merepresentasikan aspek yang berbeda sehingga tidak boleh diperlakukan sebagai ukuran tunggal performa.

Istilah metrik Lighthouse digunakan karena tidak semua metrik tersebut merupakan Core Web Vitals. Dalam penelitian ini, Lighthouse mengukur keluaran teknis pada kondisi laboratorium, sedangkan UEQ dan *Think Aloud Protocol* mengukur konstruk serta gejala yang berbeda. Pemisahan ini penting karena Wehner et al. (2023) menemukan bahwa hasil Core Web Vitals dan penilaian subjektif *web Quality of Experience* tidak selalu sejalan pada konteks studi mereka.

2.4.1 First Contentful Paint

First Contentful Paint (FCP) mengukur waktu sejak navigasi dimulai sampai peramban merender konten DOM pertama, seperti teks, gambar, SVG, atau elemen `canvas` nonputih. FCP menunjukkan kapan pengguna pertama kali memperoleh umpan balik visual bahwa halaman sedang dimuat (Google, n.d.b).

2.4.2 Largest Contentful Paint

Largest Contentful Paint (LCP) mengukur waktu sampai elemen konten terbesar yang terlihat pada viewport dirender. Metrik ini digunakan sebagai indikator waktu tampilnya konten utama, tetapi nilainya tetap dipengaruhi karakteristik elemen LCP, jalur pemuatan sumber daya, dan kondisi lingkungan pengujian (Google, n.d.c).

2.4.3 Speed Index

Speed Index mengukur kecepatan progres tampilan visual selama pemuatan halaman. Lighthouse merekam proses pemuatan dan menghitung perubahan visual antarkerangka gambar. Nilai yang lebih rendah menunjukkan bahwa bagian halaman yang terlihat disajikan lebih cepat (Google, n.d.d).

2.4.4 Total Blocking Time

Total Blocking Time (TBT) menjumlahkan bagian waktu di atas 50 milidetik dari setiap *long task* yang berlangsung antara FCP dan Time to Interactive. TBT merupakan proksi responsivitas pada pengujian laboratorium karena tugas panjang dapat menunda pemrosesan masukan pengguna. TBT bukan data Interaction to Next Paint (INP) lapangan dari populasi pengguna (Google, n.d.e).

2.4.5 Cumulative Layout Shift

Cumulative Layout Shift (CLS) mengukur kestabilan visual berdasarkan pergeseran tata letak tidak terduga. Pergeseran dapat terjadi, misalnya, ketika gambar tidak memiliki dimensi yang ditetapkan atau konten dinamis dimasukkan sebelum ruangnya disediakan. Nilai yang mendekati nol menunjukkan tampilan yang lebih stabil (Google, n.d.a).

Batas interpretasi yang digunakan sebagai konteks deskriptif ditunjukkan pada Tabel 2.2. Perbandingan utama Vue.js dan Svelte tetap dilakukan menggunakan nilai mentah dari konfigurasi Lighthouse yang sama dan tidak ditentukan hanya berdasarkan kategori ambang.

Tabel 2.2 Batas Interpretasi Metrik Performa Lighthouse

Metrik	Aspek yang Diukur	Baik	Perlu Perbaikan	Buruk
FCP	Kemunculan konten pertama	$\leq$ 1,8 detik	$>$ 1,8–3 detik	$>$ 3 detik
LCP	Tampilan konten utama	$\leq$ 2,5 detik	$>$ 2,5–4 detik	$>$ 4 detik
Speed Index	Progres tampilan visual	$\leq$ 3,4 detik	$>$ 3,4–5,8 detik	$>$ 5,8 detik
TBT	Pemblokiran main thread	$\leq$ 200 ms	$>$ 200–600 ms	$>$ 600 ms
CLS	Kestabilan visual	$\leq$ 0,1	$>$ 0,1–0,25	$>$ 0,25

Sumber: Diolah penulis berdasarkan Google (n.d.a; n.d.b; n.d.c; n.d.d; n.d.e).

2.5 Efisiensi Sumber Daya

Efisiensi sumber daya menjelaskan besarnya sumber daya yang dikirimkan dan diproses ketika aplikasi dijalankan. Penelitian ini menggunakan ukuran berkas JavaScript, total ukuran data yang ditransfer, dan jumlah permintaan jaringan. Ukuran JavaScript berkaitan dengan data yang perlu diunduh, diparsing, dan dieksekusi oleh peramban. Total ukuran transfer mencakup seluruh sumber daya jaringan,

sedangkan jumlah permintaan menunjukkan banyaknya transaksi sumber daya selama pemuatan halaman.

Ketiga metrik tersebut tidak berdiri sendiri. Ukuran transfer dapat dipengaruhi kompresi dan penyimpanan cache, sedangkan jumlah permintaan tidak langsung menunjukkan biaya setiap permintaan. Oleh karena itu, pengukuran dilakukan dengan kebijakan cache, dataset, aset, route, dan kondisi jaringan yang sama. Gambar eksternal dipertahankan sebagai sumber variansi bersama dan penggunaannya dicatat sebagai keterbatasan eksperimen.

2.6 Validasi Kesetaraan Prototipe

Validasi kesetaraan diperlukan sebelum pengukuran agar perbedaan hasil tidak disebabkan oleh ketidaksamaan fitur atau kontrak data. Kesetaraan yang dimaksud adalah kesetaraan observabel, bukan kesamaan struktur kode internal. Implementasi tetap menggunakan pola yang wajar untuk Vue.js dan Svelte.

Validasi dilakukan melalui tiga tingkat pengujian:

- *unit testing* memeriksa fungsi pengolahan data dan status secara terpisah;
- *integration testing* memeriksa hubungan komponen frontend dengan kontrak API dan respons yang ditampilkan; dan
- *end-to-end testing* menjalankan alur pengguna melalui peramban untuk memeriksa perilaku sistem secara menyeluruh.

Vitest digunakan untuk pengujian unit dan integrasi komponen (Vitest, n.d.), sedangkan Playwright digunakan untuk otomasi alur peramban (Microsoft, n.d.). Azizullah, Rahayudi, dan Priyambadha (2026) menjelaskan bahwa pengujian end-to-end mensimulasikan skenario pengguna dari awal hingga akhir dan bahwa Playwright mendukung otomasi pada beberapa peramban. Pada penelitian ini, hasil validasi menjadi prasyarat pengujian performa dan bukan variabel hasil yang digunakan untuk menentukan framework yang lebih baik.

2.7 User Experience dan Usability

ISO 9241-210:2019 menempatkan pengalaman pengguna sebagai persepsi dan respons yang muncul dari penggunaan atau penggunaan yang diantisipasi terhadap sistem, produk, atau layanan. Pengalaman tersebut dapat mencakup emosi, preferensi, persepsi, respons fisik dan psikologis, perilaku, serta pencapaian sebelum, selama, dan setelah penggunaan (ISO, 2019). Dengan demikian, pengalaman pengguna lebih luas daripada keberhasilan menyelesaikan tugas.

ISO 9241-11:2018 menjelaskan usability sebagai hasil penggunaan sistem, produk, atau layanan untuk mencapai tujuan tertentu dengan efektivitas, efisiensi, dan kepuasan dalam konteks penggunaan yang ditetapkan (ISO, 2018). Pada penelitian ini, pengalaman pengguna diukur menggunakan User Experience Questionnaire (UEQ), sedangkan hambatan saat menyelesaikan tugas diidentifikasi melalui *Think Aloud Protocol*.

Tinjauan sistematis Mortazavi et al. (2024) mengidentifikasi lebih dari 300 dimensi UX yang dikelompokkan ke dalam 13 kategori utama, antara lain usability, utility, hedonis, emosi, dan sensoris. Temuan tersebut memperkuat bahwa UX merupakan konstruk multidimensi yang tidak dapat direduksi menjadi kecepatan pemuatan atau responsivitas teknis saja. Dalam penelitian ini, UEQ menangkap penilaian pada enam skala pengalaman, sedangkan *Think Aloud Protocol* memberi bukti kontekstual mengenai penyebab keraguan, kesalahan, ekspektasi yang tidak terpenuhi, dan kegagalan tugas.

2.7.1 User Experience Questionnaire

UEQ merupakan instrumen diferensial semantik yang mengukur pengalaman pengguna melalui 26 pasangan istilah pada tujuh tingkat jawaban. Instrumen menghasilkan enam skala, yaitu *Attractiveness*, *Perspicuity*, *Efficiency*, *Dependability*, *Stimulation*, dan *Novelty* (Schrepp, 2023). Handasari, Wulandari, dan Haikal (2024) menggunakan UEQ berbahasa Indonesia dan melaporkan keenam skala secara deskriptif dalam evaluasi aplikasi JKN. Setiap responden pada penelitian ini menggunakan kedua prototipe dalam urutan yang diseimbangkan dan mengisi UEQ setelah menyelesaikan tugas pada masing-masing prototipe agar jawaban merepresentasikan kesan langsung terhadap antarmuka yang digunakan.

Enam skala tersebut menangkap aspek pengalaman yang berbeda. *Attractiveness* mengukur kesan umum terhadap produk dan merupakan dimensi valensi murni. *Perspicuity*, *Efficiency*, dan *Dependability* membentuk kualitas pragmatis yang berkaitan dengan kemudahan memahami, menyelesaikan tugas secara efisien, serta rasa kendali saat berinteraksi. *Stimulation* dan *Novelty* membentuk kualitas hedonis yang berkaitan dengan ketertarikan, motivasi, dan persepsi inovasi. Setiap skala perlu dilaporkan secara terpisah karena UEQ penuh tidak dirancang untuk digabungkan menjadi satu skor tunggal (Schrepp, 2023).

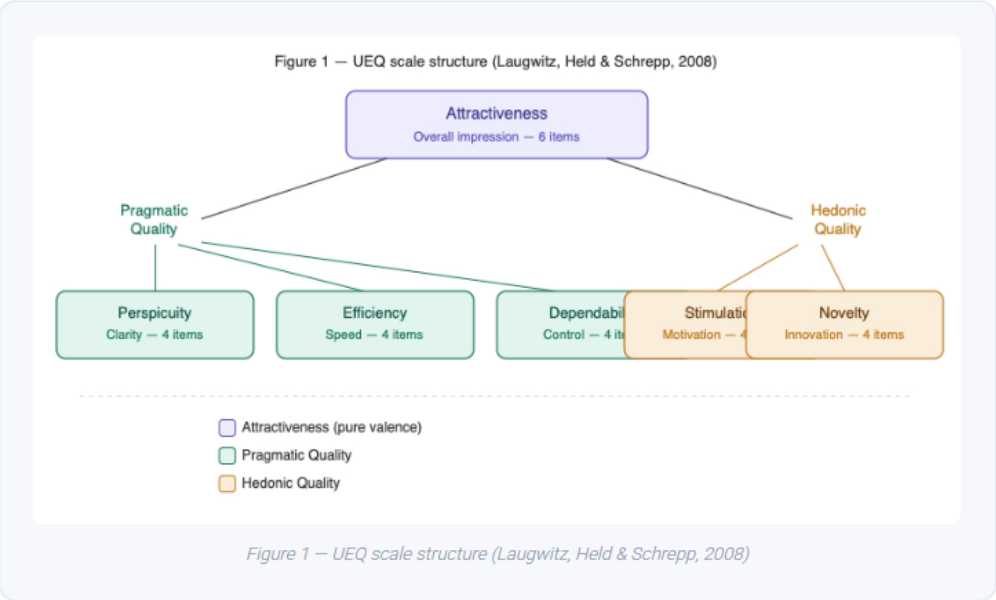
SCALE	TYPE	WHAT IT MEASURES	ITEMS
Attractiveness	PURE VALENCE	Overall impression: pleasantness, general appeal	6
Perspicuity	PRAGMATIC	Ease of learning and immediate comprehension	4
Efficiency	PRAGMATIC	Speed of task completion without unnecessary effort	4
Dependability	PRAGMATIC	Sense of control and predictability of interaction	4
Stimulation	HEDONIC	Interest, motivation and enjoyment in use	4
Novelty	HEDONIC	Perceived innovation and creativity of the product	4

Gambar 2.4 Ringkasan Skala dan Aspek yang Diukur oleh UEQ

Sumber: Fusco (2026).

Struktur UEQ menempatkan *Attractiveness* sebagai penilaian umum, sedangkan lima skala lainnya menjelaskan kualitas pragmatis dan hedonis secara lebih

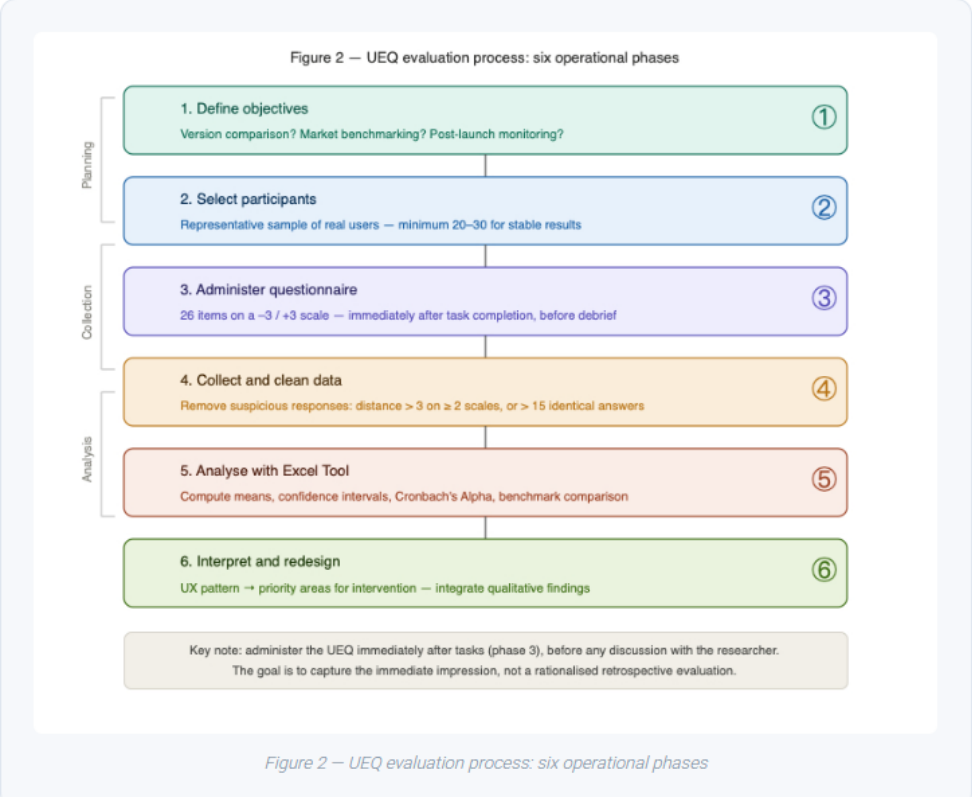
terperinci. Skala *Attractiveness* terdiri atas enam butir; masing-masing skala lain terdiri atas empat butir. Pembagian tersebut membantu membedakan pengalaman yang berhubungan dengan keberhasilan mencapai tujuan penggunaan dari pengalaman yang berhubungan dengan daya tarik dan kebaruan produk.



Gambar 2.5 Struktur Enam Skala pada UEQ

Sumber: Fusco (2026).

Menurut Handbook UEQ, evaluasi dimulai dengan penetapan tujuan, pemilihan partisipan yang sesuai dengan pengguna sasaran, dan pemberian kuesioner segera setelah tugas selesai serta sebelum diskusi dengan peneliti. Jawaban kemudian diperiksa kualitasnya, dikonversi dari rentang 1–7 ke -3 sampai $+3$ dengan memperhatikan polaritas butir, lalu diringkas menggunakan alat analisis UEQ. Hasil skala, interval kepercayaan, konsistensi internal, dan perbandingan terhadap *benchmark* dapat digunakan sebagai dasar interpretasi dan perbaikan desain (Schrepp, 2023).



Gambar 2.6 Alur Evaluasi UEQ

Sumber: Fusco (2026).

Angka 20–30 responden yang tampak pada ilustrasi alur merupakan panduan praktis yang disampaikan oleh Fusco (2026) untuk kestabilan ringkasan hasil, bukan batas minimum universal. Dalam penelitian ini, jumlah responden yang menilai kedua prototipe ditentukan berdasarkan tujuan evaluasi, ketelitian ringkasan deskriptif, dan kelayakan pelaksanaan.

Skor UEQ merepresentasikan penilaian subjektif dan tidak menggantikan metrik performa teknis. Sebaliknya, metrik Lighthouse juga tidak dapat digunakan sebagai pengganti penilaian pengalaman pengguna. Keduanya digunakan bersama temuan *Think Aloud Protocol* untuk menjawab pertanyaan penelitian yang berbeda dan menilai apakah keunggulan teknis pada suatu framework juga disertai pengalaman penggunaan yang lebih baik.

2.7.2 Think Aloud Protocol

Think Aloud Protocol meminta partisipan mengutarakan apa yang sedang diperhatikan, dipahami, diharapkan, dan dirasakan saat mengerjakan tugas. Metode tersebut membantu mengidentifikasi alasan di balik keraguan, kesalahan, atau kegagalan yang tidak selalu tampak dari skor akhir (Vanicek dan Popelka, 2023).

Penelitian menggunakan *concurrent Think Aloud Protocol*, yaitu verbalisasi dilakukan bersamaan dengan pelaksanaan tugas. Moderator dapat memberikan pengingat netral ketika partisipan berhenti berbicara, tetapi tidak mengarahkan tin-

dakan. Ucapan, aktivitas layar, status penyelesaian tugas, dan catatan observasi digunakan untuk mengelompokkan masalah berdasarkan tugas, aspek antarmuka, frekuensi, dan dampaknya.

2.8 Statistika Deskriptif dan Inferensial

Statistika deskriptif digunakan untuk merangkum hasil pengukuran, sedangkan statistika inferensial digunakan untuk menilai bukti terhadap perbedaan pada populasi pengukuran yang didefinisikan. Populasi pada pengujian performa adalah seluruh kemungkinan hasil pengulangan terhadap versi aplikasi, perangkat, peramban, dataset, dan konfigurasi eksperimen yang ditetapkan. Pengulangan teknis pada satu perangkat tidak mewakili jumlah perangkat atau pengguna yang berbeda.

2.8.1 Ukuran Pemusatan dan Penyebaran

Mean dan simpangan baku menunjukkan rata-rata serta variasi antarobservasi. Median dan *interquartile range* (IQR) memberikan ringkasan yang lebih tahan terhadap nilai ekstrem dan distribusi miring. Nilai minimum dan maksimum melengkapi informasi mengenai rentang hasil. Selisih absolut dan relatif digunakan untuk menunjukkan besar perbedaan, sehingga hasil tidak hanya dinilai dari signifikansi statistik.

2.8.2 Analisis Pengukuran Berpasangan

Setiap blok pengujian Lighthouse memuat satu run Vue.js dan satu run Svelte pada lingkungan yang sama. Kedua run diberi ID pasangan sebelum analisis; urutan framework dibalik secara bergantian antarpasangan untuk mengurangi pengaruh urutan dan perubahan lingkungan sepanjang pengujian. Parameter utama adalah mean selisih dalam pasangan. Untuk pasangan ke-i, selisih dan estimasinya didefinisikan pada Persamaan 2.1.

$$d_i = X_{\text{Vue},i} - X_{\text{Svelte},i}, \qquad \widehat{\Delta} = \bar{d} = \frac{1}{n}\sum_{i=1}^{n} d_i \tag{2.1}$$

Nilai $\widehat{\Delta} > 0$ menunjukkan mean metrik Vue.js lebih tinggi daripada Svelte, sedangkan nilai negatif menunjukkan mean Vue.js lebih rendah. Kondisi perangkat, peramban, viewport, jaringan, cache, dataset, aset, route, dan versi build dikendalikan. Pemasangan didasarkan pada blok pengambilan data, bukan pada pengurutan nilai atau penyamaan nomor run dari dua batch terpisah.

2.8.3 Pemilihan Uji dan Pelaporan

Normalitas diperiksa pada distribusi selisih pasangan d_i menggunakan Shapiro–Wilk dan Q–Q plot. Jika selisih cukup mendekati normal, mean selisih diuji dengan uji t berpasangan dua arah. Jika tidak, digunakan uji permutasi berpasangan dua arah dengan pertukaran label di dalam setiap pasangan. Holt

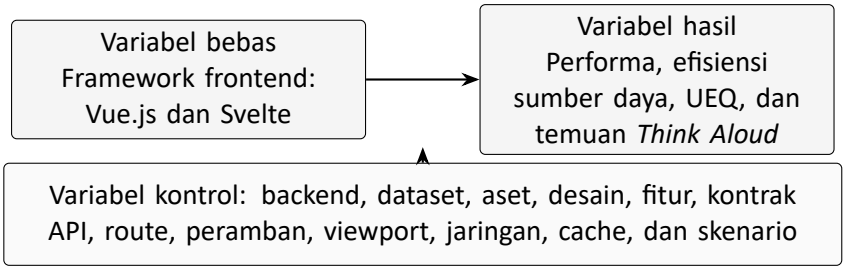
dan Sullivan (2023) membahas permutasi khusus untuk pasangan yang mempertahankan hubungan antarobservasi. Interval kepercayaan mean selisih dihitung melalui *bootstrap percentile* yang mengambil ulang pasangan utuh, sedangkan $d_z = \bar{d}/s_d$ melaporkan ukuran efek terstandar bila simpangan baku selisih tidak nol (Shatz, 2024; SciPy, n.d.).

Nilai *p-value* tidak menunjukkan probabilitas bahwa hipotesis nol benar dan tidak menunjukkan besar perbedaan. Oleh karena itu, hasil dilaporkan bersama selisih absolut atau relatif, interval kepercayaan 95%, serta ukuran efek (Phillips et al., 2022). Karena lima metrik Lighthouse diuji sebagai satu keluarga hipotesis, prosedur Holm digunakan untuk mengendalikan *family-wise error rate* (Guillermou et al., 2024). Rincian prosedur pengumpulan data, hipotesis, dan aturan keputusan dijelaskan pada Bab 3.

2.9 Kerangka Konseptual Penelitian

Landasan kepustakaan membentuk hubungan konseptual antara perlakuan, hasil, dan variabel kontrol. Framework frontend menjadi variabel bebas dengan dua perlakuan, yaitu Vue.js dan Svelte. Performa pemuatan, efisiensi sumber daya, serta pengalaman pengguna menjadi kelompok variabel hasil. Backend, dataset, aset, desain, fitur, kontrak API, route, peramban, viewport, jaringan, kebijakan cache, dan skenario penggunaan dikendalikan agar perbandingan berfokus pada implementasi frontend.

Hubungan tersebut ditunjukkan pada Gambar 2.7.



Gambar 2.7 Kerangka Konseptual Perbandingan Vue.js dan Svelte

Dengan demikian, kesimpulan tidak menentukan framework terbaik secara umum, tetapi dibatasi pada prototipe, versi build, dataset, skenario, dan lingkungan uji. Performa dan pengalaman pengguna dianalisis terpisah, lalu dipertimbangkan bersama.

BAB 3 METODOLOGI PENELITIAN

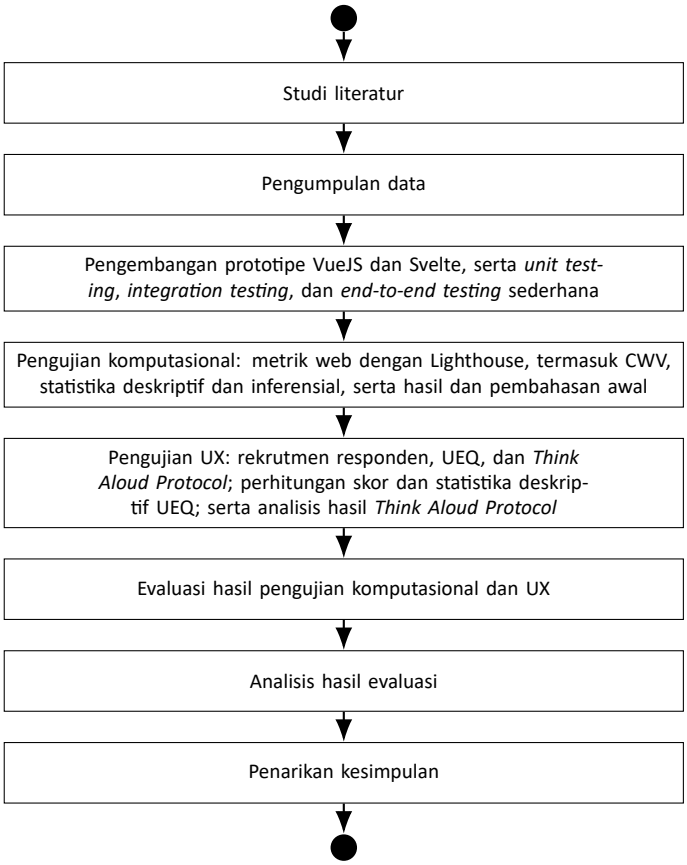
3.1 Rancangan Penelitian

Penelitian ini merupakan penelitian implementatif dengan pendekatan eksperimen komparatif. Dua prototipe website listing properti dikembangkan secara paralel menggunakan framework frontend VueJS dan Svelte dengan fitur, desain, dataset, aset, dan service backend yang dibuat setara. Setelah kesetaraan divalidasi, pengujian performa komputasional dan evaluasi pengalaman pengguna dilakukan terhadap kedua prototipe dengan kondisi yang disamakan di dalam masing-masing jenis pengujian. Lingkungan backend kedua jenis pengujian berbeda: Lighthouse menggunakan API dan basis data lokal, sedangkan evaluasi pengalaman pengguna menggunakan backend yang dideploy pada Cloudflare Workers dan Cloudflare D1.

Alur umum penelitian terdiri atas delapan tahap berikut:

1. Studi literatur.
2. Pengumpulan data.
3. Pengembangan prototipe VueJS dan Svelte, serta *unit testing*, *integration testing*, dan *end-to-end testing* sederhana.
4. Pengujian komputasional melalui pengujian metrik web dengan Lighthouse, termasuk *Core Web Vitals* (CWV), perhitungan statistika deskriptif dan inferensial, serta penyusunan hasil dan pembahasan awal.
5. Pengujian UX melalui rekrutmen responden, UEQ, dan *Think Aloud Protocol*, dilanjutkan dengan perhitungan skor dan statistika deskriptif UEQ serta analisis hasil *Think Aloud Protocol*.
6. Evaluasi hasil pengujian komputasional dan UX.
7. Analisis hasil evaluasi.
8. Penarikan kesimpulan.

Diagram alir tahapan penelitian ditunjukkan pada Gambar 3.1.



Gambar 3.1 Diagram Alir Tahapan Penelitian

3.2 Persiapan dan Pengembangan Prototipe

Tahap persiapan dan pengembangan dilakukan dengan menyiapkan tiga bagian utama, yaitu frontend VueJS, frontend Svelte, dan service backend ElysiaJS. Kedua frontend dikembangkan sebagai prototipe website responsif listing properti dengan fitur yang sama, yaitu halaman daftar properti, halaman detail properti, pencarian, *advanced filter*, sorting, pagination atau *load more*, dan wishlist sederhana. Backend ElysiaJS digunakan sebagai sumber data yang sama agar kedua frontend mengakses endpoint, dataset, dan struktur response yang setara. Eden Treaty digunakan sebagai *type-safe API client* agar kontrak tipe antara frontend dan backend terjaga secara konsisten.

Pada tahap ini, tampilan kedua frontend disesuaikan agar memiliki layout, stylesheet, konten, aset, dan alur penggunaan yang sebanding, terutama pada viewport desktop yang digunakan dalam pengujian performa. Prinsip yang diterapkan adalah *functional equivalence, bukan implementation equivalence*: kode VueJS dan Svelte tidak harus identik, tetapi perilaku yang terlihat oleh pengguna harus setara.

Variabel kontrol yang diterapkan pada kedua prototipe ditunjukkan pada Tabel 3.1.

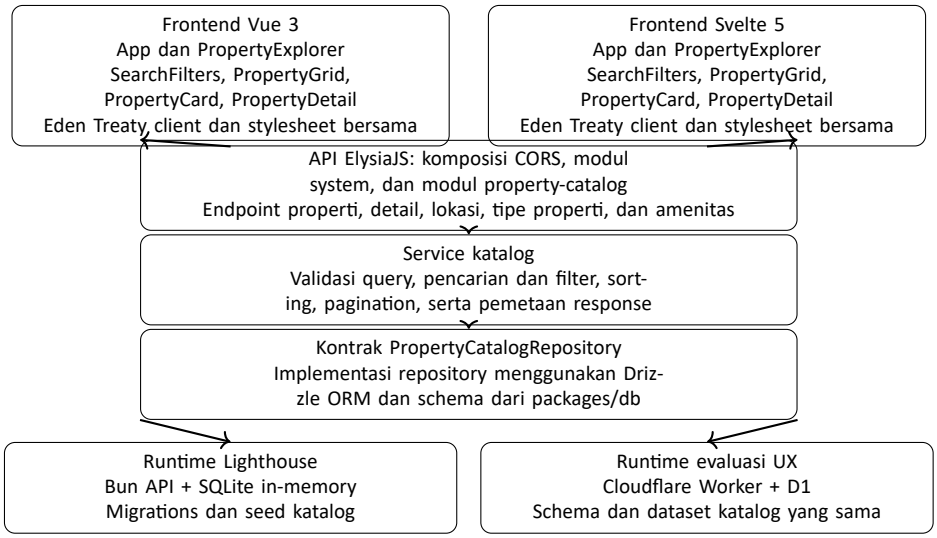
Tabel 3.1 Variabel Kontrol Penelitian

Variabel Kontrol	Ketentuan
Backend	Service backend ElysiaJS yang sama
Dataset	Dataset properti yang sama
Aset	File gambar dan sumber daya yang sama
Desain	Layout dan stylesheet yang sama
Fitur	Fungsionalitas yang setara
Browser	Browser dan versi yang sama
Viewport	Ukuran viewport yang sama
Skenario	Interaksi pengujian yang sama

Dengan persiapan tersebut, perbandingan yang dilakukan dapat difokuskan pada perbedaan karakteristik VueJS dan Svelte sebagai framework frontend, bukan pada perbedaan data, fitur, desain, atau backend yang digunakan.

3.2.1 Arsitektur Pengembangan Prototipe

Prototipe dikembangkan dengan arsitektur *client–server* modular di dalam satu monorepo. Frontend VueJS dan Svelte berjalan sebagai aplikasi terpisah, tetapi memakai kontrak API, stylesheet, dataset, dan alur fitur yang sama. Arsitektur aplikasi dan dua runtime backend yang digunakan dalam penelitian ditunjukkan pada Gambar 3.2.



Gambar 3.2 Arsitektur Aplikasi dan Runtime Backend Prototipe

Pada sisi frontend, masing-masing aplikasi memasang komponen `PropertyExplorer` sebagai pengatur state katalog, filter, hasil pencarian, detail, status pemuatan, error, dan wishlist sesi. Komponen tersebut menggunakan `catalog.ts` untuk membaca dan memperbarui query URL serta meminta opsi katalog, daftar properti, dan detail melalui API client Eden Treaty. Tipe `App` yang diekspor backend menjadi sumber kontrak client; kedua frontend

juga memuat stylesheet global dari paket workspace yang sama. Komponen form, grid, kartu, dan detail membagi tanggung jawab tampilan tanpa mengubah perilaku yang diamati pengguna.

Pada sisi backend, fungsi `createApp` menggabungkan konfigurasi CORS dengan modul sistem dan modul katalog. Modul katalog menangani endpoint HTTP dan meneruskan permintaan ke service. Service memvalidasi parameter, menjalankan pencarian, filter, sorting dan pagination, lalu menyusun response. Akses data dipisahkan melalui kontrak `PropertyCatalogRepository`; implementasi repository mengambil snapshot tabel katalog melalui Drizzle ORM. Skema tabel, migrasi, dan seed deterministik ditempatkan di paket `packages/db`.

Kontrak repository yang sama digunakan pada dua runtime. Untuk Lighthouse, API dijalankan dengan Bun dan repository membaca SQLite in-memory yang diinisialisasi dari file migrasi serta seed katalog. Untuk evaluasi UX, aplikasi Elysia dijalankan pada Cloudflare Workers dengan Cloudflare D1 sebagai binding database. Dengan demikian, pengujian UX dan Lighthouse berbeda pada infrastruktur runtime dan database, tetapi menggunakan modul API, kontrak data, serta perilaku katalog yang sama. Gambar properti dibaca dari URL aset yang tercatat pada dataset; gambar tidak dilayani oleh endpoint API. Aplikasi React yang terdapat di workspace berfungsi sebagai referensi parity dan tidak menjadi perlakuan penelitian.

SQLite digunakan pada lingkungan lokal karena berjalan di dalam proses aplikasi sebagai pustaka dan tidak memerlukan server basis data terpisah. Hal ini menyederhanakan penyiapan lingkungan Lighthouse; basis data in-memory dapat dibentuk ulang dari migrasi dan seed katalog deterministik. Gaffney et al. (2022) menjelaskan bahwa SQLite bersifat mandiri dan dirancang untuk mendukung operasi aplikasi seperti membaca serta memperbarui data. Pada prototipe ini, SQLite menyediakan data katalog yang digunakan pada daftar, pencarian, filter, dan detail properti. Pemilihan ini didasarkan pada kecocokan praktis dengan kebutuhan prototipe dan pengujian terkendali, bukan klaim bahwa SQLite lebih cepat daripada sistem basis data lain. Untuk evaluasi UX, Cloudflare D1 menggunakan query engine SQLite dan mendukung sebagian besar konvensi SQL SQLite (Cloudflare, 2026), sehingga skema katalog yang sama dapat digunakan pada kedua runtime. Kompatibilitas tersebut tidak berarti semua fitur SQLite didukung D1.

3.2.2 Struktur Tabel Basis Data

Skema basis data didefinisikan menggunakan Drizzle ORM pada paket `packages/db` dan menargetkan SQLite yang kompatibel dengan Cloudflare D1. Migrasi membentuk struktur tabel, sedangkan `catalog.sql` mengisi dataset deterministik yang sama pada lingkungan pengujian. Seluruh kolom pada skema bersifat wajib diisi; nilai bawaan dan kunci khusus dijelaskan pada tabel. Tipe boolean disimpan sebagai `INTEGER` oleh SQLite dan dipetakan menjadi boolean oleh Drizzle.

Tabel referensi menyimpan lokasi, jenis properti, host, dan amenitas. Rincian kolomnya ditunjukkan pada Tabel 3.2.

Tabel 3.2 Struktur Tabel Referensi Basis Data Prototipe

Tabel	Kolom dan tipe SQLite	Fungsi dan kendala
`locations`	`id` TEXT (PK); `city`, `province`, `country`, dan `display_name` TEXT.	Menyimpan data lokasi. Kolom `city` memiliki indeks untuk mendukung pencarian. Satu lokasi dapat digunakan oleh banyak properti.
`property_types`	`id` TEXT (PK); `name` dan `slug` TEXT.	Menyimpan kategori properti. Nilai `slug` bersifat unik dan dipakai untuk mengenali tipe secara stabil.
`hosts`	`id` TEXT (PK); `name` dan `avatar_url` TEXT; `joined_year` INTEGER; `is_superhost` INTEGER (boolean, default `false`).	Menyimpan informasi host yang ditampilkan pada halaman detail. Satu host dapat terkait dengan beberapa properti.
`amenities`	`id` TEXT (PK); `name` dan `icon_name` TEXT.	Menyimpan daftar fasilitas. Nama amenitas bersifat unik dan fasilitas dihubungkan ke properti melalui `property_amenities`.

Tabel properti dan tabel relasi menyimpan atribut listing serta hubungan antara properti dengan gambar, fasilitas, dan status awal wishlist. Rinciannya ditunjukkan pada Tabel 3.3.

Tabel 3.3 Struktur Tabel Properti dan Relasi Basis Data Prototipe

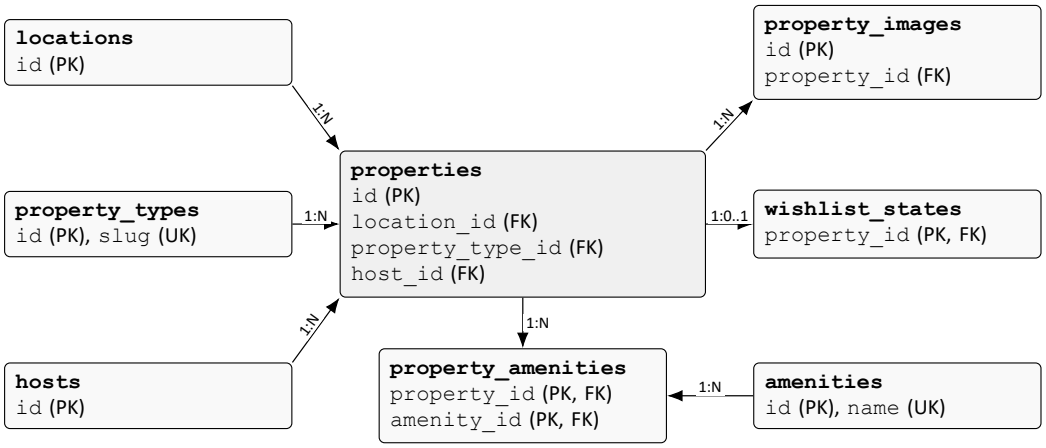
Tabel	Kolom dan tipe SQLite	Fungsi dan kendala
`properties`	`id` TEXT (PK); `title`, `slug`, `description`, `created_at`, dan `updated_at` TEXT; `location_id`, `property_type_id`, dan `host_id` TEXT (FK); `price_per_night`, `review_count`, `max_guests`, `bedrooms`, `beds`, dan `bathrooms` INTEGER; `rating` REAL; `is_guest_favorite` INTEGER (boolean, default `false`). `review_count` memiliki default 0.	Tabel inti data properti. Foreign key menghubungkan setiap properti ke satu lokasi, tipe, dan host dengan aturan `ON DELETE RESTRICT`. `slug` unik; indeks tersedia untuk lokasi, tipe, harga, dan rating.

Tabel 3.3 (lanjutan)

Tabel	Kolom dan tipe SQLite	Fungsi dan kendala
property_images	id TEXT (PK); property_id TEXT (FK); image_url dan alt_text TEXT; sort_order INTEGER (default 0); is_cover INTEGER (boolean, default false).	Menyimpan gambar dan urutannya. Setiap gambar milik satu properti; penghapusan properti menghapus gambarnya. Indeks menggabungkan property_id dan sort_order.
property_amenities	property_id dan amenity_id TEXT (FK); keduanya membentuk primary key gabungan.	Tabel penghubung many-to-many antara properti dan amenitas. Penghapusan properti atau amenitas menghapus relasinya; indeks tersedia pada amenity_id.
wishlist_states	property_id TEXT (PK dan FK); is_wishlisted INTEGER (boolean, default false).	Menyediakan nilai awal wishlist untuk setiap properti. Penghapusan properti menghapus baris terkait. Toggle wishlist selama sesi tidak menulis perubahan ke tabel ini.

Dengan foreign key tersebut, satu lokasi, tipe properti, atau host dapat berhubungan dengan banyak baris pada properties; satu properti dapat memiliki banyak gambar; dan relasi banyak-ke-banyak properti–amenitas direpresentasikan oleh property_amenities. Tabel wishlist_states hanya menyimpan nilai awal dari seed, sedangkan perubahan wishlist oleh pengguna dikelola pada state frontend. Fitur *load more* juga tidak memerlukan tabel tersendiri karena menggunakan parameter halaman dan metadata pagination dari API.

Ringkasan relasi dan kunci antar tabel ditunjukkan pada Gambar 3.3. Atribut lengkap setiap tabel tetap ditampilkan pada Tabel 3.2 dan Tabel 3.3.



Gambar 3.3 ERD Tabel Katalog Properti

PK berarti *primary key*, FK berarti *foreign key*, dan UK berarti kunci unik. Label 1:N menunjukkan bahwa satu baris tabel induk dapat berhubungan dengan nol atau lebih baris tabel anak; label 1:0..1 pada wishlist menunjukkan paling banyak satu baris untuk setiap properti. Setiap properti wajib mengacu pada tepat satu lokasi, tipe, dan host. Pada tabel penghubung amenitas, kedua kolom kunci membentuk primary key gabungan dan masing-masing menjadi foreign key ke tabel properti atau amenitas.

3.2.3 Logika Backend dan Pseudocode

Endpoint backend dikelompokkan ke dalam modul sistem dan modul katalog. Route menerima request, memanggil service, lalu mengembalikan response beserta status HTTP. Logika utama pada service adalah validasi parameter, pembacaan snapshot katalog melalui repository, pemrosesan daftar atau detail, dan pemetaan hasil ke bentuk response.

Pada endpoint daftar properti, nilai default adalah halaman 1, batas 12 properti per halaman, dan sorting rekomendasi. Parameter angka harus valid; batas halaman maksimum adalah 50, nilai kapasitas dan jumlah ruang minimal 1, harga tidak boleh negatif, dan harga maksimum tidak boleh lebih rendah daripada harga minimum. Nilai sorting dibatasi pada rekomendasi, harga naik, harga turun, atau rating turun. Parameter yang tidak valid menghasilkan HTTP 400 dengan pesan error terstruktur sebelum data katalog diproses.

Listing 3.1 Pseudocode Logika Backend Daftar Properti

```
FUNCTION GET /properties(query):
    filters = validateAndNormalize(query)
    IF filters tidak valid:
        RETURN HTTP 400 dengan error terstruktur

    catalog = repository.getSnapshot()
    hasil = []

    FOR property IN catalog.properties:
        cocok jika seluruh kondisi yang diisi terpenuhi:
            search cocok tanpa membedakan huruf pada judul,
                lokasi, atau tipe properti
            lokasi dan tipe cocok dengan pilihan query
```

```
            minPrice <= harga <= maxPrice jika batas tersedia
            kapasitas, kamar, tempat tidur, dan kamar mandi
                memenuhi nilai minimum
            semua amenitas yang diminta tersedia
        IF cocok:
            tambahkan property ke hasil

    urutkan hasil sesuai sort dengan tie-breaker deterministik
    total = jumlah hasil
    pageItems = ambil limit item mulai dari (page - 1) * limit

    RETURN HTTP 200 dengan:
        data = petakan pageItems menjadi PropertyListItem
        meta = { page, limit, total, totalPages, hasMore }
END
```

Endpoint detail mencari properti berdasarkan ID atau slug. Jika tidak ditemukan, service menghasilkan HTTP 404; jika ditemukan, data properti dipetakan bersama lokasi, tipe, host, gambar, amenitas, dan status wishlist awal. Endpoint `/locations`, `/property-types`, dan `/amenities` mengembalikan daftar referensi dari snapshot katalog. Wishlist tidak memiliki endpoint mutasi: backend menyediakan status awal, sedangkan perubahan tombol wishlist disimpan pada state lokal frontend selama sesi.

Listing 3.2 Pseudocode Backend Detail dan Data Referensi

```
FUNCTION GET /properties/{idOrSlug}:
    catalog = repository.getSnapshot()
    property = cari berdasarkan idOrSlug sebagai ID atau slug
    IF property tidak ditemukan:
        RETURN HTTP 404 dengan error PROPERTY_NOT_FOUND
    RETURN HTTP 200 dengan mapPropertyDetail(catalog, property)
END

FUNCTION GET /locations, /property-types, atau /amenities:
    catalog = repository.getSnapshot()
    RETURN daftar yang sesuai dari catalog
END
```

3.2.4 Rancangan UI/UX Prototipe

Antarmuka dirancang untuk mendukung tugas menemukan dan mengeksplorasi properti. Struktur informasi kedua prototipe dibuat sama agar perbedaan framework tidak mengubah langkah maupun informasi yang diterima pengguna. Struktur layar utama dan fungsi setiap bagiannya ditunjukkan pada Tabel 3.4.

Tabel 3.4 Struktur Antarmuka dan Fungsi UX Prototipe

Bagian antarmuka	Elemen dan fungsi
Kerangka aplikasi	Header dan tautan untuk melewati navigasi menuju konten utama; menjadi orientasi awal dan mendukung navigasi keyboard.
Katalog	Pengantar halaman, form pencarian dan filter, ringkasan hasil, status halaman, grid kartu properti, serta kontrol untuk memuat hasil berikutnya.
Kartu properti	Gambar, tipe, rating, nama, lokasi, kapasitas, harga per malam, dan tombol wishlist; gambar atau judul membuka halaman detail.
Detail properti	Tombol kembali, identitas dan galeri properti, kapasitas, deskripsi, amenitas, informasi host, ringkasan harga, serta tombol wishlist.
Status sistem	Skeleton saat pemuatan awal, pesan dan aksi coba lagi saat gagal, pesan hasil kosong beserta reset filter, dan status selesai saat tidak ada halaman lanjutan.

Form katalog memisahkan pencarian utama dari filter lanjutan. Filter lanjutan mencakup rentang harga, kapasitas, jumlah kamar tidur, tempat tidur, kamar mandi, dan amenitas. Pengguna mengirim perubahan melalui tombol pencarian; hasil tidak diperbarui pada setiap perubahan karakter. Grid properti menggunakan tiga kolom pada desktop, dua kolom pada lebar layar sampai dengan 900 piksel, dan satu kolom pada lebar sampai dengan 620 piksel. Pada halaman detail, kartu harga berada di sisi konten pada layar lebar dan mengikuti alur konten pada layar sampai dengan 900 piksel.

Label kontrol, nama aksesibel, teks alternatif gambar, fokus keyboard yang terlihat, serta pengumuman perubahan jumlah hasil disamakan pada kedua implementasi. Elemen dekoratif tidak menggantikan label teks. Tampilan pemuatan, kegagalan, hasil kosong, dan hasil lengkap diperlakukan sebagai bagian dari rancangan antarmuka agar perbandingan tidak hanya mencakup kondisi berhasil.

3.2.5 Spesifikasi Fitur dan Pseudocode

Fitur prototipe dan perilaku yang harus ekuivalen dirangkum pada Tabel 3.5. Pseudocode berikut menyatakan alur yang menjadi acuan implementasi dan validasi; implementasi aktual menggunakan sintaks serta mekanisme masing-masing framework.

Tabel 3.5 Fitur Prototipe dan Perilaku yang Divalidasi

Fitur	Perilaku yang diharapkan pada kedua prototipe
Inisialisasi katalog	Membaca filter dari URL, memuat opsi katalog, lalu meminta halaman pertama properti.
Pencarian dan filter	Menerapkan keyword, lokasi, tipe, rentang harga, kapasitas, jumlah kamar, tempat tidur, kamar mandi, dan seluruh amenitas terpilih.
Sorting	Mengirim salah satu urutan rekomendasi, harga naik, harga turun, atau rating turun ke API; perubahan memulai hasil dari halaman pertama.
Pagination / muat lebih banyak	Meminta halaman berikutnya hanya jika metadata menyatakan masih ada hasil, lalu menambahkan hasil baru ke grid.
Detail dan navigasi	Membuka detail berdasarkan slug, menyinkronkan URL dan browser history, serta mengembalikan pengguna ke katalog dengan query yang dapat dipulihkan.
Wishlist	Menginisialisasi status dari data properti dan memperbaruinya secara lokal agar status kartu dan detail konsisten selama sesi.
Status dan pemulihan	Menampilkan pemuatan, kegagalan dengan aksi coba lagi, hasil kosong dengan reset, atau hasil beserta kontrol halaman berikutnya.

Pencarian, filter, sorting, dan pagination diproses melalui kontrak API yang sama. Query divalidasi sebelum diterapkan pada katalog; beberapa amenitas yang dipilih menggunakan semantik AND. Hasil diurutkan secara deterministik sebelum pagination sehingga urutan dan batas halaman setara pada kedua frontend.

Listing 3.3 Pseudocode Inisialisasi dan Pencarian Katalog

```
ON application mount:
    IF URL berisi slug properti:
        buka detail tanpa menambah history
    ELSE:
        filters = baca filter dari query URL
        muat lokasi, tipe, dan amenitas secara paralel
        muat daftar properti(filters, page=1, append=false)

ON submit form atau reset filter:
    filters = salin draft form atau nilai default
    simpan filters ke query URL
    muat daftar properti(filters, page=1, append=false)

FUNCTION loadPropertyList(filters, page, append):
    requestId = requestId + 1
    tandai daftar sedang dimuat
    response = minta halaman dari API dengan query yang sama
    IF requestId masih merupakan request terbaru:
        IF append:
            tambahkan response.data ke daftar
        ELSE:
            ganti daftar dengan response.data
        simpan response.meta dan gabungkan status wishlist
    tampilkan error hanya jika request yang gagal masih yang terbaru
    hentikan status loading hanya untuk request terbaru

```

```
ON click "Muat lebih banyak":
    IF pagination.hasMore:
        loadPropertyList(filters, pagination.page + 1, append=true)
END
```

Listing 3.4 Pseudocode Detail, Browser History, dan Wishlist

```
ON click properti:
    simpan URL katalog pada history.state
    push URL detail berdasarkan slug
    minta detail; abaikan response jika ada request detail yang lebih baru

ON browser back atau tombol "Kembali ke hasil":
    pulihkan URL katalog dan filternya
    muat ulang halaman pertama katalog

ON toggle wishlist(propertyId):
    wishlist[propertyId] = NOT wishlist[propertyId]
    perbarui tombol wishlist pada kartu dan detail
    jangan kirim request mutasi atau menyimpan status lintas sesi
END
```

3.2.6 Perbandingan Implementasi VueJS dan Svelte

Kedua frontend menggunakan data, algoritme bisnis, kontrak API, struktur halaman, dan alur interaksi yang ekuivalen. Perbandingan implementasi berfokus pada cara masing-masing framework menyatakan komponen, state reaktif, komunikasi antarkomponen, dan lifecycle. Ringkasan padanan konstruksi framework ditunjukkan pada Tabel 3.6.

Tabel 3.6 Padanan Konstruksi Implementasi VueJS dan Svelte

Kebutuhan	VueJS	Svelte
Struktur komponen	Single File Component, `<script setup>` dan `<template>`	Komponen `.svelte` dengan script dan markup
State katalog	`shallowRef` dan assignment melalui `.value`	`$state.raw` dan assignment ulang
Nilai turunan	`computed`	`$derived`
Props dan aksi child	`defineProps` dan `defineEmits`	`$props` dan callback props
Input form	`v-model`	`bind:value` atau `bind:group`
Render kondisi dan daftar	`v-if`, `v-for`, dan `:key`	blok `{#if}`, `{#each}` berkunci, dan `{#key}`
Lifecycle dan history	Hook `onMounted` dan `onUnmounted` untuk listener browser	Hook `onMount` dan listener window bawaan Svelte

Contoh ringkas state dan komunikasi aksi dari komponen anak memperlihatkan perbedaan sintaks dengan perilaku yang sama: data daftar bereaksi ter-

hadap assignment, label hasil diturunkan dari metadata, dan aksi membuka properti diteruskan ke komponen induk.

Listing 3.5 Padanan State dan Aksi Komponen pada VueJS dan Svelte

```
VueJS:
const properties = shallowRef<Property[]>([])
const resultLabel = computed(
    () => `${pagination.value?.total ?? 0} properti ditemukan`
)
const emit = defineEmits<{ open: [slug: string] }>()
properties.value = response.data
emit("open", property.slug)

Svelte:
let properties = $state.raw<Property[]>([])
let resultLabel = $derived(
    `${pagination?.total ?? 0} properti ditemukan`
)
let { openProperty } = $props<{ openProperty: (slug: string) => void }>()
properties = response.data
openProperty(property.slug)
```

Algoritme search, filter, sorting, pagination, pemuatan detail, dan pembentukan query tidak diubah untuk menyesuaikan sintaks framework. Karena itu, tabel ini menjelaskan perbedaan bentuk implementasi, bukan hasil pengukuran kompleksitas atau klaim keunggulan salah satu framework. Kesetaraan perilaku tetap diperiksa melalui pengujian unit, integrasi, dan end-to-end pada bagian berikutnya.

3.2.7 Versi Pustaka dan Perangkat Pengembangan

Versi pustaka frontend dicatat berdasarkan versi yang terkunci di `pnpm-lock.yaml` pada workspace saat penelitian. Tabel 3.7 memuat framework, perangkat build dan pemeriksaan tipe, serta pustaka pengujian yang menjadi dependensi langsung kedua prototipe.

Tabel 3.7 Versi Pustaka dan Perangkat Frontend VueJS dan Svelte

Pustaka / perangkat	VueJS	Svelte
Framework	Vue 3.5.41	Svelte 5.56.8
Plugin Vite	`@vitejs/plugin-vue` 6.0.8	`@sveltejs/vite-plugin-svelte` 7.3.0
Build tool	Vite 8.2.1	Vite 8.2.1
Bahasa dan pemeriksaan tipe	TypeScript 5.9.3; `vue-tsc` 3.3.9	TypeScript 5.9.3; `svelte-check` 4.7.5
Test runner dan DOM	Vitest 4.1.10; jsdom 30.0.1	Vitest 4.1.10; jsdom 30.0.1
Utilitas pengujian komponen	`@vue/test-utils` 2.4.11	`@testing-library/svelte` 5.4.2; `@testing-library/user-event` 14.6.4; `@testing-library/jest-dom` 7.0.1
Pustaka API dan stylesheet bersama	Paket internal workspace	Paket internal workspace

Versi pada tabel merepresentasikan dependensi langsung yang digunakan oleh aplikasi dan pengujiannya. Pustaka API serta stylesheet merupakan paket internal workspace yang sama, bukan dependensi eksternal dengan versi terpisah. Penggunaan lockfile menjaga versi ter-resolve tetap sama saat instalasi ulang.

3.3 Validasi Kesetaraan Fitur

Validasi kesetaraan fitur dilakukan sebelum pengujian performa untuk memastikan bahwa implementasi VueJS dan Svelte memiliki fungsi, perilaku antarmuka, dan logika bisnis yang ekuivalen secara observabel. Validasi dibagi menjadi tiga tingkatan pengujian.

3.3.1 Unit Testing

Vitest digunakan untuk menguji logika pengolahan data secara terpisah pada masing-masing prototipe. Logika yang diuji meliputi pencarian, penyaringan, pengurutan, pagination, dan formatting data properti.

Untuk menjamin kesetaraan logika bisnis antar-framework, algoritme inti seperti pencarian dan penyaringan properti dibuat ekuivalen pada kedua implementasi. Algoritme penyaringan properti dirumuskan dalam bentuk pseudocode pada Kode 3.6.

Listing 3.6 Pseudocode Algoritme Penyaringan Properti Ekuivalen

```
function filterProperties(properties, query, filters):
    hasil = properties

    if query tidak kosong:
        hasil = filter berdasarkan nama atau lokasi
```

```
6
7       if priceRange tersedia:
8           hasil = filter berdasarkan rentang harga
9
10      if propertyType tersedia:
11          hasil = filter berdasarkan tipe properti
12
13      if amenities tersedia:
14          hasil = filter berdasarkan fasilitas (AND)
15
16      if guests tersedia:
17          hasil = filter berdasarkan kapasitas minimum
18
19      return hasil
```

Penerapan algoritme ekuivalen ini memastikan bahwa logika pemrosesan data bisnis yang dipakai pada VueJS dan Svelte dibuat identik, sedangkan perbedaan hanya berada pada mekanisme implementasi internal framework.

3.3.2 Integration Testing

Validasi integrasi dilakukan untuk memastikan bahwa kedua prototipe frontend berinteraksi dengan service backend ElysiaJS secara setara. Aspek yang divalidasi meliputi:

- *Request* API ke endpoint yang benar;
- *Response* data yang diterima sesuai kontrak;
- Pemrosesan data dari API;
- Hasil data yang ditampilkan pada antarmuka.

3.3.3 End-to-End Testing

Playwright digunakan untuk memastikan alur penggunaan (*user flow*) pada prototipe VueJS dan Svelte berjalan setara pada viewport desktop. Skenario pengujian kesetaraan fitur ditunjukkan pada Tabel 3.8.

Tabel 3.8 Skenario Pengujian Kesetaraan Fitur

ID	Skenario	Hasil yang Diharapkan
F01	Search properti	Kedua prototipe menghasilkan data yang setara
F02	Advanced filter	Hasil filter setara
F03	Sorting	Urutan data setara
F04	Wishlist	Perubahan status wishlist berjalan setara
F05	Detail properti	Informasi properti yang ditampilkan setara
F06	Load more	Pemuatan batch data tambahan berjalan setara

Apabila ditemukan perbedaan hasil atau perilaku, implementasi disesuaikan terlebih dahulu hingga kedua frontend memenuhi seluruh kriteria penerimaan.

3.4 Pengujian Performa Komputasional

Pengujian performa komputasional dilakukan setelah implementasi VueJS dan Svelte dinyatakan memiliki fitur dan perilaku yang setara. Kedua aplikasi terlebih dahulu dijalankan dalam mode *production build* agar pengukuran dilakukan terhadap versi aplikasi yang telah melalui proses optimasi build.

3.4.1 Lingkungan Pengujian

Seluruh pengujian performa komputasional dilakukan pada lingkungan yang dikontrol agar hasil pengukuran dapat dibandingkan secara adil. Konfigurasi lingkungan pengujian ditunjukkan pada Tabel 3.9.

Tabel 3.9 Konfigurasi Lingkungan Pengujian

Parameter	**Konfigurasi**
Perangkat keras	AMD Ryzen 7 4800H (8 Cores, 16 Threads @ 2.9 GHz, up to 4.2 GHz), RAM 16 GB DDR4, NVIDIA GeForce RTX 3050 Laptop GPU
Sistem operasi	Windows 11 64-bit
Browser	Google Chrome 151.0.7922.138
Infrastruktur backend	ElysiaJS REST API lokal pada mesin pengujian
Infrastruktur database	SQLite lokal dalam memori dengan dataset deterministik yang sama
API client contract	Eden Treaty (Type-safe client)
Viewport	Desktop 1350×940 piksel dengan *device scale factor* 1
CPU throttling	$1\times$ slowdown sesuai *preset desktop* Lighthouse
Network throttling	Simulasi *Dense 4G* sesuai *preset desktop* Lighthouse (RTT 40 ms dan throughput 10.240 Kbps)
Cache policy	*Fresh browser context* pada setiap observasi
Build mode	Production build
Unit eksperimen	Satu observasi Lighthouse pada salah satu framework dalam kondisi pengujian yang dikendalikan
Pemasangan run	Satu blok memuat satu run VueJS dan satu run Svelte dengan ID pasangan yang sama; urutan framework dibalik bergantian antarblok
Jumlah pengulangan	20 blok lengkap pada route utama; masing-masing blok memuat satu run per framework
Versi Lighthouse	Lighthouse CI 0.15.1
Versi Playwright	Playwright 1.62.1

Pengujian dilakukan pada kondisi perangkat, browser, viewport, dataset, aset, serta skenario penggunaan yang sama sehingga perbedaan hasil pengukuran dapat lebih difokuskan pada karakteristik implementasi masing-masing framework. Kedua frontend dijalankan sebagai *production preview* lokal dan mengakses API ElysiaJS lokal dengan basis data SQLite dalam memori yang diisi dari dataset deterministik yang sama. Dengan demikian, hasil Lighthouse mencerminkan lingkungan laboratorium lokal yang dikendalikan, bukan latensi layanan Cloudflare.

3.4.2 Pengujian Metrik Performa Google Lighthouse

Google Lighthouse digunakan untuk mengukur performa pemuatan kedua prototipe pada viewport desktop dalam mode *production build*. Metrik utama yang diukur adalah First Contentful Paint (FCP), Largest Contentful Paint (LCP), Speed Index, Total Blocking Time (TBT), dan Cumulative Layout Shift (CLS). Kelima metrik tersebut digunakan untuk membandingkan kecepatan munculnya konten awal, kecepatan tampilnya konten utama, progres visual halaman, waktu pemblokiran main thread, dan stabilitas visual kedua prototipe.

Sebelum pengukuran berpasangan, skrip memeriksa secara otomatis bahwa lima alur tugas UX pada Subbab 3.5.2 dapat diselesaikan pada kedua prototipe dengan API lokal. Pemeriksaan ini hanya memastikan kesiapan alur dan tidak menjadi observasi Lighthouse maupun pengganti evaluasi pengguna. Report Lighthouse untuk analisis inferensial tetap berasal dari pemuatan route utama.

Prosedur pengujian dilakukan dengan alur berikut:

1. Membangun *production build* pada kedua prototipe.
2. Menjalankan aplikasi pada lingkungan pengujian yang telah dikonfigurasi.
3. Menjalankan satu blok pilot untuk memeriksa kesiapan server, pengambilan report, dan pencatatan ID pasangan; jumlah data utama ditetapkan 20 blok sebelum pengambilan data.
4. Menyiapkan kondisi pengujian yang sama, meliputi browser, viewport, throttling, cache policy, route, dataset, dan versi build.
5. Menjalankan n blok Lighthouse; setiap blok terdiri atas satu run VueJS dan satu run Svelte. Urutan VueJS–Svelte dan Svelte–Vue dibalik secara bergantian antarblok.
6. Menjaga kondisi pengujian konstan pada seluruh run dan mencatat waktu, framework, route, browser, serta versi build untuk menilai kemungkinan variasi lingkungan.
7. Menggunakan *fresh browser context* pada setiap observasi dan menjalankan Google Lighthouse tanpa interaksi pengguna lain pada perangkat pengujian.
8. Menyimpan nilai mentah FCP, LCP, Speed Index, TBT, dan CLS beserta ID pasangan, urutan framework, waktu, route, browser, dan versi build dari setiap observasi.

9. Mengulangi pengujian hingga mencapai n run yang telah ditetapkan pada setiap framework dan skenario.

Run pilot tidak digabungkan ke dalam data utama. Observasi hanya dikeluarkan apabila terjadi kegagalan sistem, browser, jaringan, atau instrumen berdasarkan kriteria eksklusi yang ditetapkan sebelum analisis. Nilai ekstrem tidak dihapus hanya karena memengaruhi signifikansi dan tetap dilaporkan sebagai bagian dari variasi pengujian apabila tidak terdapat bukti kegagalan pengukuran.

3.4.3 Pengujian Efisiensi Sumber Daya

Pengujian efisiensi sumber daya dilakukan menggunakan hasil *production build* dan Chrome DevTools, khususnya melalui Network Panel. Metrik yang diamati meliputi:

- **JavaScript bundle size**: ukuran kode JavaScript yang perlu dikirim dan diproses oleh browser.
- **Total transferred size**: keseluruhan data yang ditransfer melalui jaringan ketika halaman dimuat.
- **Request count**: jumlah permintaan sumber daya yang dilakukan oleh browser.

Seluruh pengukuran dilakukan pada halaman, dataset, aset, browser, dan kondisi jaringan yang sama. Dengan demikian, perbandingan efisiensi sumber daya dapat digunakan sebagai data pendukung untuk menjelaskan perbedaan karakteristik performa komputasional antara VueJS dan Svelte.

3.5 Evaluasi User Experience

Pada evaluasi pengalaman pengguna, kedua prototipe mengakses backend ElysiaJS yang dideploy melalui Cloudflare Workers dengan basis data Cloudflare D1. Endpoint, kontrak API, dan dataset dibuat sama untuk kelompok VueJS dan Svelte. Perbedaan lingkungan backend dari pengujian Lighthouse dicatat saat menafsirkan hasil kedua jenis evaluasi.

3.5.1 Partisipan Penelitian

Evaluasi *User Experience* melibatkan responden yang mewakili target pengguna prototipe website listing properti. Kriteria inklusi meliputi pengguna yang memiliki pengalaman mengakses website melalui perangkat mobile dan desktop serta pernah menggunakan Airbnb atau layanan pemesanan akomodasi daring sejenis, seperti Traveloka, Agoda, atau tiket.com, untuk mencari atau membandingkan tempat menginap. Pengalaman melakukan pemesanan tidak disyaratkan karena tugas pada prototipe terbatas pada pencarian dan pemilihan properti. Kriteria tersebut diperiksa melalui pertanyaan penyaringan sebelum sesi evaluasi. Metode pengambilan sampel ditetapkan sebelum pelaksanaan evaluasi.

Schrepp, Hinderks, dan Thomaschewski (2017) menyatakan bahwa sekitar 20–30 pengguna biasanya menghasilkan pengukuran UEQ yang cukup stabil.

Kushendriawan et al. (2021) juga menggunakan rentang 20–30 responden sebagai acuan dalam evaluasi UEQ. Berdasarkan kedua rujukan tersebut, penelitian ini menetapkan target operasional sekurang-kurangnya 30 responden dengan data UEQ valid untuk kedua prototipe. Setiap responden mengevaluasi VueJS dan Svelte sehingga tersedia sedikitnya 30 penilaian per prototipe yang berpasangan menurut responden. Angka tersebut bukan batas minimum universal yang menjamin ketelitian hasil pada semua penelitian UEQ.

3.5.2 Skenario Pengujian

Evaluasi dilakukan pada dua tipe perangkat, yaitu mobile dan desktop. Setiap tipe dibagi menjadi subtipe *high-end* dan *low-end*, sehingga terdapat empat strata perangkat. Definisi operasional strata ditunjukkan pada Tabel 3.10.

Tabel 3.10 Strata Perangkat Evaluasi User Experience

Kode	Tipe	Kriteria Perangkat	Konfigurasi yang Dikendalikan
MH	Mobile *high-end*	Android kelas *upper-mid/flagship*, RAM minimal 8 GB, SoC berusia maksimal tiga tahun	Chrome dengan versi mayor yang sama, orientasi potret, *refresh rate* 60 Hz jika tersedia, dan mode hemat daya dinonaktifkan
ML	Mobile *low-end*	Android kelas *entry/lower-mid*, RAM maksimal 4 GB	Chrome dengan versi mayor yang sama, orientasi potret, *refresh rate* 60 Hz, dan mode hemat daya dinonaktifkan
DH	Desktop *high-end*	Windows 11, minimal 8 *logical processor*, RAM minimal 16 GB, dan penyimpanan SSD	Chrome dengan versi mayor yang sama, zoom 100%, jendela dimaksimalkan, dan proses latar belakang non-esensial ditutup
DL	Desktop *low-end*	Windows 11, 2–4 *logical processor*, RAM 4–8 GB, dan penyimpanan SSD	Chrome dengan versi mayor yang sama, zoom 100%, jendela dimaksimalkan, dan proses latar belakang non-esensial ditutup

Model perangkat, CPU atau SoC, RAM, versi sistem operasi dan browser, resolusi, viewport, serta *device pixel ratio* dicatat pada setiap sesi. Seluruh strata menggunakan jaringan pengujian yang sama. High-end dan low-end diperlakukan sebagai strata konteks penggunaan, sehingga perbedaan antarkelas perangkat tidak ditafsirkan sebagai pengaruh CPU saja.

Setiap responden ditempatkan pada satu strata perangkat dan menggunakan kedua prototipe pada perangkat yang sama. Target 30 responden dialokasikan sebanyak 7–8 orang pada setiap strata perangkat; urutan penggunaan kedua prototipe diseimbangkan di dalam setiap strata. Kedua kelompok urutan menjalankan rangkaian tugas yang sama pada masing-masing prototipe. Instruksi kepada responden menyatakan tujuan yang perlu dicapai tanpa menyebut nama fitur, letak kontrol, atau urutan tindakan. Tujuan tugas meliputi:

1. Menemukan akomodasi yang sesuai dengan lokasi dan jumlah tamu.
2. Menemukan properti yang memenuhi beberapa kebutuhan harga, tipe, kapasitas, dan fasilitas.
3. Menemukan pilihan dengan harga per malam paling murah.
4. Mencari properti, menyimpannya ke *wishlist*, lalu kembali ke halaman daftar dan memeriksa status simpanannya.
5. Menemukan pilihan lain yang belum tampak pada daftar awal melalui *load more*.

Kelima tujuan tersebut dituangkan menjadi tugas TA-01 sampai TA-05 pada instrumen evaluasi UX. Skenario *load more* hanya dijalankan apabila dataset berisi lebih banyak properti daripada nilai *limit* halaman. Kondisi tersebut harus diterapkan secara identik pada VueJS dan Svelte sebelum pengambilan data.

3.5.3 Prosedur Pengujian

Pengujian menggunakan desain berpasangan (*within-subject*) untuk faktor framework: setiap responden mencoba kedua prototipe. Prototipe A adalah VueJS dan Prototipe B adalah Svelte; nama framework tidak disebutkan kepada responden selama sesi. Faktor perangkat tetap antarkelompok karena setiap responden ditempatkan pada satu dari empat strata. Sebelum sesi, 30 responden diacak ke dua kelompok urutan dengan target 15 orang per kelompok dan jumlah yang diseimbangkan di dalam setiap strata sebagai berikut:

- **Kelompok A**: responden menjalankan tugas pada Prototipe A, mengisi UEQ untuk Prototipe A, lalu menjalankan tugas pada Prototipe B dan mengisi UEQ untuk Prototipe B.
- **Kelompok B**: responden menjalankan tugas pada Prototipe B, mengisi UEQ untuk Prototipe B, lalu menjalankan tugas pada Prototipe A dan mengisi UEQ untuk Prototipe A.

Pembalikan urutan digunakan untuk membatasi pengaruh belajar dan kelelahan saat tugas serupa diulang. Di antara kedua prototipe, responden diberi jeda singkat dan kondisi awal browser, pencarian, filter, serta wishlist dikembalikan ke keadaan yang sama. Urutan prototipe dan kondisi perangkat dicatat pada setiap sesi agar pengaruh urutan dapat dipertimbangkan saat menafsirkan hasil.

Selama menjalankan tugas pada masing-masing prototipe, responden dipersilakan menyampaikan apa yang mereka pikirkan atau rasakan secara

spontan sambil berfokus menyelesaikan tujuan tugas. Sesi dilaksanakan melalui pertemuan daring menggunakan Zoom atau pertemuan langsung. Ucapan yang tersedia, perilaku yang teramati, kesulitan, dan status penyelesaian setiap tugas dicatat terpisah menurut prototipe. UEQ diisi segera setelah rangkaian tugas pada setiap prototipe selesai, sebelum responden beralih ke prototipe berikutnya atau berdiskusi dengan moderator. Skenario, data, tampilan, instruksi, dan kondisi penggunaan dibuat setara pada kedua prototipe.

3.5.4 User Experience Questionnaire

User Experience Questionnaire (UEQ) diberikan kepada responden segera setelah menyelesaikan rangkaian tugas pada setiap prototipe dan sebelum beralih ke prototipe lain atau berdiskusi dengan moderator. Setiap responden mengisi dua UEQ, masing-masing untuk satu prototipe, agar jawaban merepresentasikan kesan langsung terhadap antarmuka yang baru digunakan. Penelitian menggunakan UEQ penuh versi Bahasa Indonesia yang terdiri atas 26 pasangan istilah dengan skala diferensial semantik tujuh tingkat (Schrepp, 2023).

Responden memilih satu nilai dari 1 sampai 7 pada setiap butir. Nilai 1 menunjukkan kesan yang paling dekat dengan istilah di sisi kiri, nilai 7 paling dekat dengan istilah di sisi kanan, dan nilai 4 berada di tengah. Urutan butir dan posisi istilah kiri–kanan dipertahankan karena sebagian butir menempatkan istilah positif di kiri dan sebagian lainnya di kanan. Butir UEQ yang digunakan ditunjukkan pada Tabel 3.11.

Tabel 3.11 Butir User Experience Questionnaire Versi Bahasa Indonesia

No.	Istilah Kiri	Skala	Istilah Kanan
1	Menyusahkan	1–7	Menyenangkan
2	Tak dapat dipahami	1–7	Dapat dipahami
3	Kreatif	1–7	Monoton
4	Mudah dipelajari	1–7	Sulit dipelajari
5	Bermanfaat	1–7	Kurang bermanfaat
6	Membosankan	1–7	Mengasyikkan
7	Tidak menarik	1–7	Menarik
8	Tak dapat diprediksi	1–7	Dapat diprediksi
9	Cepat	1–7	Lambat
10	Berdaya cipta	1–7	Konvensional
11	Menghalangi	1–7	Mendukung
12	Baik	1–7	Buruk
13	Rumit	1–7	Sederhana
14	Tidak disukai	1–7	Menggembirakan
15	Lazim	1–7	Terdepan
16	Tidak nyaman	1–7	Nyaman
17	Aman	1–7	Tidak aman
18	Memotivasi	1–7	Tidak memotivasi
19	Memenuhi ekspektasi	1–7	Tidak memenuhi ekspektasi
20	Tidak efisien	1–7	Efisien
21	Jelas	1–7	Membingungkan

Tabel 3.11 (lanjutan)

No.	Istilah Kiri	Skala	Istilah Kanan
22	Tidak praktis	1–7	Praktis
23	Terorganisasi	1–7	Berantakan
24	Atraktif	1–7	Tidak atraktif
25	Ramah pengguna	1–7	Tidak ramah pengguna
26	Konservatif	1–7	Inovatif

Hasil UEQ dihitung pada enam skala, yaitu *Attractiveness* (butir 1, 12, 14, 16, 24, dan 25), *Perspicuity* (butir 2, 4, 13, dan 21), *Efficiency* (butir 9, 20, 22, dan 23), *Dependability* (butir 8, 11, 17, dan 19), *Stimulation* (butir 5, 6, 7, dan 18), serta *Novelty* (butir 3, 10, 15, dan 26). Jawaban mentah disimpan dalam rentang 1–7, kemudian dikonversi ke rentang -3 sampai $+3$ menggunakan alat analisis resmi UEQ agar polaritas setiap butir diproses dengan benar. Skor setiap skala diperoleh dari rata-rata butir yang membentuk skala tersebut untuk setiap responden pada masing-masing prototipe. Dengan demikian, enam skor skala menjadi unit analisis; UEQ tidak dijumlahkan menjadi satu skor total dan tidak dikonversi menjadi skor System Usability Scale (SUS).

Nilai UEQ pada prototipe VueJS dan Svelte diringkas secara deskriptif dari pasangan penilaian responden yang sama, baik secara keseluruhan maupun pada setiap strata perangkat dan kelompok urutan. Analisis per strata dan urutan digunakan untuk menunjukkan konteks penggunaan; tidak dilakukan uji signifikansi perbedaan framework atau perangkat dari data UEQ.

3.5.5 Think Aloud Protocol

Pengujian tugas disertai kesempatan untuk melakukan verbalisasi secara bersamaan (*concurrent think aloud*). Responden diberi kebebasan untuk mengutarakan tanggapan selama menyelesaikan tujuan tugas, tanpa kewajiban menjelaskan setiap tindakan secara terus-menerus. Karena intensitas verbalisasi berbeda antarresponden, data verbal diperlakukan sebagai pelengkap observasi perilaku dan hasil tugas, bukan sebagai ukuran keberhasilan tersendiri.

Moderator menyampaikan tujuan setiap tugas tanpa memperagakan antarmuka atau menyebut langkah penyelesaiannya. Pelaksanaan secara daring dan langsung dicatat sebagai konteks sesi. Untuk setiap tugas, peneliti mencatat bukti yang tersedia berupa:

- ucapan responden serta tindakan yang teramati pada layar;
- kesalahan, pengulangan tindakan, dan bagian antarmuka yang menghambat penyelesaian tugas;
- status penyelesaian tugas, yaitu berhasil tanpa hambatan, berhasil dengan hambatan, atau tidak berhasil; dan
- catatan waktu, moda sesi, dan konteks ketika permasalahan terjadi.

3.6 Teknik Analisis Data

3.6.1 Analisis Performa Komputasional

Hasil pengujian berulang dianalisis menggunakan statistik deskriptif dan inferensial. Statistik deskriptif yang dilaporkan untuk setiap framework, skenario, dan metrik meliputi jumlah observasi, mean, simpangan baku, median, minimum, maksimum, dan *interquartile range* (IQR). Mean dan simpangan baku digunakan untuk menunjukkan rata-rata serta variasi antar-run, sedangkan median dan IQR digunakan sebagai ringkasan yang lebih tahan terhadap nilai ekstrem.

Mean dan simpangan baku sampel dihitung menggunakan Persamaan 3.1 dan Persamaan 3.2.

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \tag{3.1}$$

$$s = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}} \tag{3.2}$$

$$\text{IQR} = Q_3 - Q_1 \tag{3.3}$$

Perbandingan inferensial dilakukan pada selisih dua run di dalam blok yang sama. Setiap blok memiliki ID pasangan, satu run dari masing-masing framework, dan kondisi route, build, browser, viewport, throttling, cache policy, API, dataset, serta aset yang sama. Selisih pasangan dan mean selisih dinyatakan pada Persamaan 3.4.

$$d_i = x_{\text{Vue},i} - x_{\text{Svelte},i}, \qquad \Delta = \bar{d} = \frac{1}{n}\sum_{i=1}^{n} d_i \tag{3.4}$$

Nilai $\Delta > 0$ menunjukkan nilai mean VueJS lebih tinggi daripada Svelte, sedangkan $\Delta < 0$ menunjukkan sebaliknya. Karena FCP, LCP, Speed Index, TBT, dan CLS menggunakan prinsip nilai lebih rendah lebih baik, arah selisih harus dibaca bersama karakteristik metrik. Hipotesis dua arah untuk setiap metrik dirumuskan menggunakan mean populasi μ sebagai berikut:

$$\begin{aligned} H_0 &: \mu_{\text{Vue}} - \mu_{\text{Svelte}} = 0 \\ H_1 &: \mu_{\text{Vue}} - \mu_{\text{Svelte}} \neq 0 \end{aligned} \tag{3.5}$$

Hipotesis nol menyatakan bahwa tidak terdapat perbedaan mean populasi pengukuran antara VueJS dan Svelte pada kondisi eksperimen yang ditetapkan. Tingkat signifikansi ditetapkan sebesar $\alpha = 0{,}05$ sebelum data utama dianalisis.

Normalitas diperiksa pada selisih pasangan d_i menggunakan uji Shapiro–Wilk dan *Q–Q plot*. Nilai $p > 0{,}05$ ditafsirkan sebagai kegagalan menolak hipotesis normalitas, bukan bukti bahwa data pasti normal. Pemilihan uji komparatif ditunjukkan pada Tabel 3.12.

Tabel 3.12 Pemilihan Uji Komparatif Metrik Performa

Kondisi Selisih Pasangan	Uji	Pelaporan
Selisih pasangan mendekati normal	Uji t berpasangan dua arah	Mean selisih, interval kepercayaan 95%, dan d_z
Selisih pasangan tidak normal atau konstan	Uji permutasi berpasangan dua arah	Mean selisih, interval kepercayaan *bootstrap* 95%, dan d_z bila terdefinisi

Uji homogenitas varians antarkelompok tidak digunakan sebagai prasyarat karena unit inferensi adalah selisih di dalam pasangan. Uji permutasi menukar label VueJS dan Svelte hanya di dalam tiap pasangan, setara dengan membalik tanda d_i. Prosedur ini mengandaikan label dapat dipertukarkan di dalam pasangan di bawah hipotesis nol. Urutan framework diseimbangkan untuk membatasi pengaruh urutan; variasi antarblok tetap diperiksa saat menafsirkan hasil.

Karena lima metrik Lighthouse diuji sebagai satu keluarga hipotesis, keputusan signifikansi disesuaikan menggunakan prosedur Holm untuk mengendalikan *family-wise error rate*. Hasil tidak hanya ditentukan berdasarkan *p-value*; setiap metrik juga dilaporkan bersama selisih absolut dan persentase, interval kepercayaan 95%, serta ukuran efek. Dengan demikian, signifikansi statistik dapat dibedakan dari relevansi praktis perbedaan performa.

Jika selisih pasangan mendekati normal, uji t berpasangan menghitung statistik dari mean dan simpangan baku selisih seperti pada Persamaan 3.6; derajat bebasnya $n - 1$. Interval kepercayaan 95% mean selisih dihitung melalui *bootstrap percentile* dengan 20.000 pengambilan ulang pasangan utuh. Ukuran efek d_z menyatakan mean selisih relatif terhadap simpangan baku selisih dan tidak dilaporkan bila simpangan baku selisih nol.

$$t = \frac{\bar{d}}{s_d/\sqrt{n}} \tag{3.6}$$

$$d_z = \frac{\bar{d}}{s_d}, \qquad df = n - 1 \tag{3.7}$$

Pada kondisi nonnormal, uji permutasi berpasangan dua arah membentuk distribusi nol dengan membalik tanda tiap selisih pasangan secara acak sebanyak 100.000 kali. Nilai p adalah proporsi mean selisih simulasi yang nilai absolutnya setidaknya sebesar mean selisih observasi. Pasangan tidak dipecah atau dipasangkan ulang; hasil dilaporkan bersama mean selisih, interval kepercayaan *bootstrap*, dan d_z bila terdefinisi.

Prosedur Holm diterapkan dengan mengurutkan lima nilai p mentah dari yang terkecil. Untuk nilai urutan ke-j dari $m = 5$ hipotesis, nilai yang disesuaikan dihitung menurut Persamaan 3.8, kemudian dibatasi maksimum satu dan dibuat tidak menurun mengikuti urutan nilai p. Keputusan akhir menggunakan $p_{\text{Holm}} < 0{,}05$.

$$p_{\text{Holm},(j)} = \min\left(1, \max_{k \le j}\{(m-k+1)p_{(k)}\}\right) \tag{3.8}$$

Notebook analisis menyimpan ID pasangan, nilai mentah, selisih pasangan, hasil pemeriksaan asumsi, jenis uji, statistik uji, p mentah, p Holm, interval kepercayaan, dan ukuran efek. Inferensi hanya diterapkan pada blok yang memiliki kedua report lengkap. Kesimpulan dibatasi pada konfigurasi eksperimen yang dikendalikan dan tidak digeneralisasi sebagai perbedaan pada seluruh perangkat atau pengguna.

Jumlah pengulangan ditetapkan sebanyak 20 blok sebelum pengambilan data utama. Pilot digunakan untuk memeriksa kelancaran prosedur dan tidak digabungkan dengan 20 blok utama. Jumlah blok tidak ditambah atau dihentikan berdasarkan hasil *p-value* sementara. Pengulangan pada satu perangkat merupakan replikasi teknis yang meningkatkan ketelitian untuk lingkungan tersebut, tetapi tidak dianggap mewakili jumlah perangkat atau pengguna yang berbeda.

JavaScript bundle size yang deterministik dilaporkan sebagai nilai eksak dari artefak *production build* dan tidak diuji secara inferensial. Total transferred size dan request count dilaporkan secara deskriptif; analisis inferensial hanya diterapkan jika keduanya dikumpulkan sebagai observasi berulang dari kedua kelompok dengan prosedur eksperimen yang sama.

Hasil pengukuran FCP, LCP, Speed Index, TBT, dan CLS dibandingkan terhadap *threshold* Lighthouse yang ditunjukkan pada Tabel 3.13.

Tabel 3.13 Threshold Metrik Performa Lighthouse

Metrik	Good	Needs Improvement	Poor
FCP	$\leq$ 1,8 detik	$>$ 1,8–3 detik	$>$ 3 detik
LCP	$\leq$ 2,5 detik	$>$ 2,5–4 detik	$>$ 4 detik
Speed Index	$\leq$ 3,4 detik	$>$ 3,4–5,8 detik	$>$ 5,8 detik
TBT	$\leq$ 200 ms	$>$ 200–600 ms	$>$ 600 ms
CLS	$\leq$ 0,1	$>$ 0,1–0,25	$>$ 0,25

Ringkasan aspek, metrik, dan instrumen pengukuran yang digunakan pada penelitian ini ditunjukkan pada Tabel 3.14.

Tabel 3.14 Instrumen dan Alat Pengukuran

Aspek	Alat / Metode
FCP, LCP, Speed Index, TBT, dan CLS	Google Lighthouse
JavaScript bundle size	Hasil production build dan Chrome DevTools
Total transferred size	Chrome DevTools Network
Request count	Chrome DevTools Network
User Experience kuantitatif	User Experience Questionnaire (UEQ)
User Experience kualitatif	*Think Aloud Protocol*, rekaman layar dan suara, serta lembar observasi
Kesetaraan fungsi	Vitest dan Playwright

3.6.2 Analisis UEQ

Hasil UEQ dihitung pada enam skala, yaitu *Attractiveness*, *Perspicuity*, *Efficiency*, *Dependability*, *Stimulation*, dan *Novelty*. Untuk setiap prototipe dan setiap skala, statistik deskriptif yang dilaporkan meliputi jumlah responden (n), mean, simpangan baku, median, *interquartile range* (IQR), minimum, dan maksimum. Mean dan simpangan baku menunjukkan kecenderungan penilaian serta variasi antarresponden, sedangkan median dan IQR melengkapi ringkasan ketika sebaran penilaian tidak simetris atau dipengaruhi nilai ekstrem.

Karena setiap responden menilai kedua prototipe, perbandingan deskriptif disajikan sebagai selisih skor setiap skala dalam pasangan responden, $d_i = x_{i,\text{Vue}} - x_{i,\text{Svelte}}$, lalu diringkas dengan mean selisih $\bar{d}_{\text{UEQ}}$. Untuk setiap skala dilaporkan mean selisih dan arah perbedaannya tanpa uji signifikansi. Nilai $\bar{d}_{\text{UEQ}} > 0$ menunjukkan penilaian UEQ yang lebih tinggi pada VueJS, sedangkan nilai negatif menunjukkan penilaian lebih tinggi pada Svelte. Karena nilai UEQ yang lebih tinggi menunjukkan pengalaman pengguna yang lebih positif, arah ini berbeda dari sebagian besar metrik performa komputasional yang menggunakan prinsip nilai lebih rendah lebih baik.

Hasil deskriptif disajikan untuk setiap prototipe sebagai pelaporan utama dan, sebagai pelengkap, untuk setiap strata perangkat dan kelompok urutan. Interpretasi menggunakan enam skala UEQ dan, jika digunakan, kategori *benchmark* dari alat resmi UEQ sebagai konteks deskriptif. Kategori *benchmark* tidak menggantikan nilai mentah, ukuran penyebaran, maupun selisih dalam pasangan. Hasil UEQ tidak dihitung sebagai skor SUS karena kedua instrumen memiliki butir dan aturan penskoran yang berbeda.

3.6.3 Analisis Think Aloud Protocol

Ucapan responden yang tersedia ditranskripsikan pada bagian yang relevan dengan pelaksanaan tugas. Bukti verbal dipisahkan dari perilaku yang teramati dan status penyelesaian tugas. Ucapan tentang keraguan atau ekspektasi yang tidak terpenuhi serta perilaku seperti kesalahan, pengulangan tindakan, atau kegagalan menyelesaikan tugas dicatat sebagai temuan sesuai sumber buktinya. Ketiadaan

ucapan tidak ditafsirkan sebagai ketiadaan masalah ataupun sebagai bukti bahwa responden mengalami kebingungan. Temuan serupa kemudian digabungkan dan dikelompokkan berdasarkan tugas serta aspek antarmuka, seperti navigasi, pemahaman label dan konten, umpan balik sistem, kontrol pencarian dan penyaringan, serta tampilan visual.

Setiap temuan dibandingkan berdasarkan jumlah responden yang mengalaminya pada masing-masing prototipe dan diklasifikasikan menurut dampaknya:

- **Kritis**: menghalangi responden menyelesaikan tugas.
- **Mayor**: menyebabkan kesalahan atau kesulitan berarti, tetapi responden masih dapat menyelesaikan tugas setelah melakukan pemulihan.
- **Minor**: menyebabkan keraguan atau ketidaknyamanan tanpa menghalangi penyelesaian tugas.

Hasil analisis disajikan dalam matriks temuan yang memuat tugas, kategori masalah, bukti ucapan atau observasi, jumlah responden yang mengalami masalah pada setiap prototipe, dan tingkat dampak. Perbedaan intensitas verbalisasi serta moda sesi daring dan langsung dipertimbangkan sebagai keterbatasan saat menafsirkan temuan kualitatif; banyaknya ucapan seorang responden tidak dihitung sebagai banyaknya responden yang mengalami masalah.

3.7 Analisis Komparatif Hasil Pengujian

Tahap analisis komparatif dilakukan untuk membandingkan hasil dari seluruh aspek pengujian secara keseluruhan guna menjawab rumusan masalah penelitian. Aspek yang dibandingkan meliputi:

- metrik performa Google Lighthouse (FCP, LCP, Speed Index, TBT, dan CLS);
- efisiensi sumber daya (JavaScript bundle size, total transferred size, request count);
- *User Experience* kuantitatif (hasil UEQ pada enam skala); dan
- *User Experience* kualitatif (temuan *Think Aloud Protocol*).

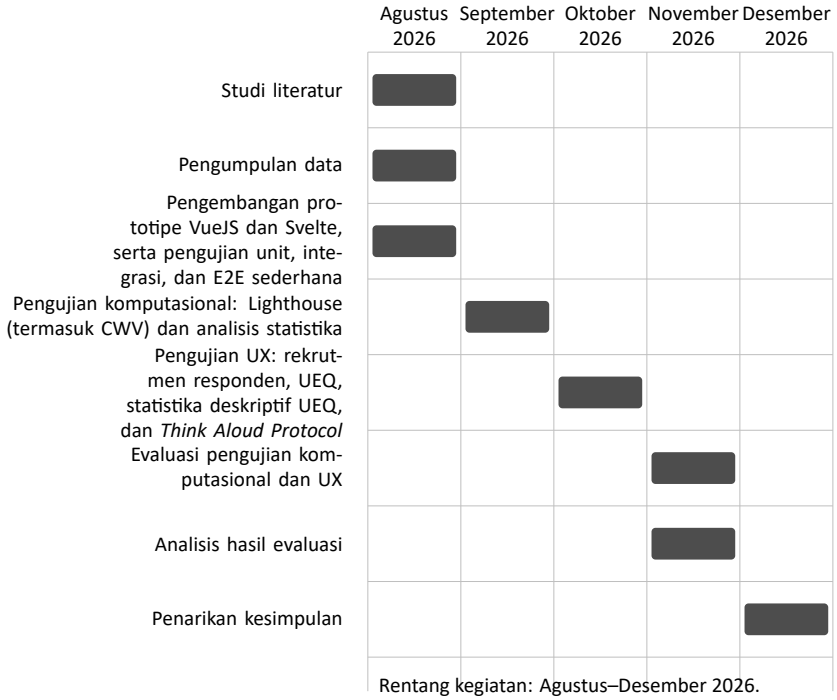
Hasil perbandingan tersebut digunakan untuk menganalisis bagaimana perbedaan performa komputasional dan pengalaman pengguna antara VueJS dan Svelte dapat menjadi pertimbangan dalam pemilihan framework frontend untuk pengembangan website listing yang kaya konten visual dan memiliki fitur interaktif. Kesimpulan performa mempertimbangkan nilai pusat, variasi, selisih absolut dan relatif, interval kepercayaan, ukuran efek, signifikansi setelah koreksi Holm, serta kategori *threshold* Lighthouse. Penelitian tidak menetapkan framework terbaik secara umum dan membatasi kesimpulan pada prototipe, versi build, skenario, serta lingkungan eksperimen yang diuji.

Hasil performa dan pengalaman pengguna tidak dilebur menjadi satu skor gabungan. Arah kedua hasil dibandingkan untuk melihat apakah keduanya konvergen atau justru berbeda. Nilai Lighthouse yang lebih baik tidak secara otomatis ditafsirkan sebagai UX yang lebih baik tanpa dukungan UEQ dan temuan *Think*

Aloud Protocol; demikian pula, penilaian UX tidak digunakan untuk menggantikan penjelasan mengenai karakteristik performa teknis.

3.8 Jadwal Penelitian

Jadwal penelitian disusun berdasarkan rentang kegiatan akademik pada semester berjalan. Tanggal akhir setiap rentang digunakan sebagai batas penyelesaian internal agar implementasi, pengujian, analisis, dan penyusunan laporan selesai sebelum tahapan penilaian berikutnya. Perkiraan timeline pengerjaan ditunjukkan pada Gambar 3.4.



Gambar 3.4 Timeline Pengerjaan Skripsi

Sumber: Diolah penulis berdasarkan rencana penelitian, 2026.

Jadwal tersebut bersifat perkiraan dan dapat disesuaikan dengan hasil konsultasi dosen pembimbing serta kesiapan pengumpulan data dan responden.

BAB 4 HASIL

4.1 Hasil Pengujian Performa Komputasional

4.1.1 Cakupan Data Pengujian

Analisis menggunakan 20 blok pengujian berpasangan pada halaman utama (route /). Setiap blok berisi satu report Vue.js dan satu report Svelte dari *production build*; urutan framework dibalik selang-seling antarblok. Dengan demikian, terdapat 20 observasi per framework dan 20 selisih pasangan untuk setiap metrik. Report pilot tidak masuk analisis. Notebook memvalidasi kelengkapan pasangan, URL, *runtime error*, dan nilai audit sebelum menghitung hasil.

Kedua frontend menggunakan API, dataset, aset, route, browser, viewport, kebijakan cache, dan kondisi pengujian yang sama. Data ini merupakan pengulangan teknis pada satu konfigurasi laboratorium. Metrik yang diolah ialah FCP, LCP, Speed Index, TBT, CLS, ukuran transfer JavaScript, total transfer, dan jumlah permintaan. Waktu dinyatakan dalam milidetik, transfer dalam byte, sedangkan CLS tanpa satuan. Lighthouse hanya mengukur pemuatan halaman utama setelah pemeriksaan alur tugas; nilai ini bukan pengukuran durasi masing-masing tugas.

LCP dan CLS termasuk Core Web Vitals. TBT merupakan proksi responsivitas laboratorium, bukan INP lapangan. Selisih dinyatakan sebagai $d_i = x_{\text{Vue},i} - x_{\text{Svelte},i}$; nilai positif berarti Vue.js lebih tinggi. Pada lima metrik performa, nilai yang lebih rendah lebih baik. Hasil hanya berlaku untuk kondisi pengujian ini dan tidak mewakili seluruh perangkat atau populasi pengguna.

4.1.2 Statistika Deskriptif Metrik Lighthouse

Mean, simpangan baku sampel (SD), median, kuartil, IQR, minimum, dan maksimum dihitung dari nilai mentah setiap framework. Tabel 4.1 dan Tabel 4.2 menyajikan ringkasannya. Pembulatan dilakukan hanya untuk tampilan tabel; seluruh uji menggunakan nilai penuh.

Tabel 4.1 Statistika Deskriptif Metrik Lighthouse: Pemusatan dan Penyebaran

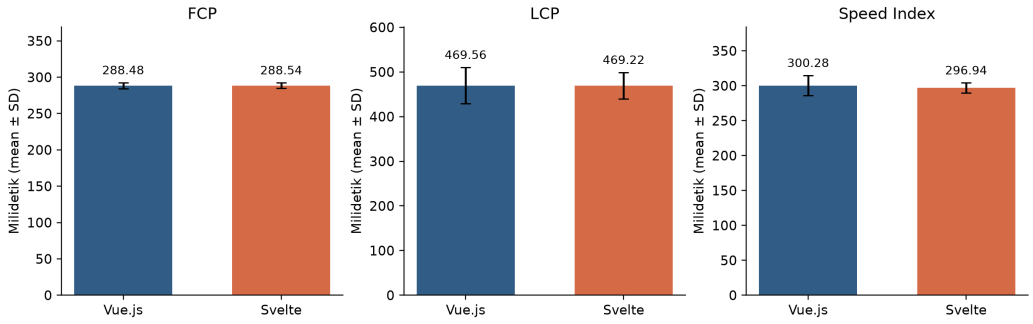
Metrik	Framework	n	Mean	SD	Median	IQR
FCP	Vue.js	20	288,479	3,999	287,900	2,041
FCP	Svelte	20	288,538	3,785	287,429	1,941
LCP	Vue.js	20	469,563	40,798	479,954	76,456
LCP	Svelte	20	469,218	29,786	476,882	21,279
Speed Index	Vue.js	20	300,278	14,488	294,852	6,917
Speed Index	Svelte	20	296,943	7,221	296,246	8,640
TBT	Vue.js	20	0,000	0,000	0,000	0,000
TBT	Svelte	20	0,000	0,000	0,000	0,000
CLS	Vue.js	20	0,000587	0,000000	0,000587	0,000000
CLS	Svelte	20	0,000587	0,000000	0,000587	0,000000

Tabel 4.2 Statistika Deskriptif Metrik Lighthouse: Kuartil dan Rentang

Metrik	Framework	Q_1	Q_3	Minimum	Maksimum
FCP	Vue.js	286,990	289,031	284,781	304,521
FCP	Svelte	286,520	288,462	285,228	299,355
LCP	Vue.js	425,716	502,172	399,941	516,394
LCP	Svelte	461,252	482,531	409,587	516,862
Speed Index	Vue.js	292,171	299,089	287,094	338,075
Speed Index	Svelte	292,240	300,880	285,228	311,989
TBT	Vue.js	0,000	0,000	0,000	0,000
TBT	Svelte	0,000	0,000	0,000	0,000
CLS	Vue.js	0,000587	0,000587	0,000587	0,000587
CLS	Svelte	0,000587	0,000587	0,000587	0,000587

Mean FCP Vue.js sebesar 288,479 ms dan Svelte 288,538 ms, sehingga selisih Vue–Svelte sebesar $-0{,}059$ ms. Mean LCP masing-masing 469,563 ms dan 469,218 ms (selisih 0,345 ms). Mean Speed Index masing-masing 300,278 ms dan 296,943 ms (selisih 3,334 ms). Selisih kecil ini perlu dibaca bersama variasi antarblok dan hasil inferensial, bukan sebagai bukti keunggulan yang pasti. Semua nilai TBT adalah nol dan semua nilai CLS identik pada kedua framework.

Gambar 4.1 memperlihatkan mean dan SD tiga metrik waktu yang bervariasi. TBT dan CLS tidak ditampilkan sebagai batang karena nilainya identik dan sudah tercantum dalam tabel.



Gambar 4.1 Perbandingan Mean dan SD FCP, LCP, serta Speed Index
Sumber: Diolah penulis dari keluaran notebook analisis.

Tabel 4.3 menunjukkan selisih mean serta persentase terhadap mean Vue.js. Untuk metrik dengan mean nol pada kedua framework, persentase tidak terdefinisi dan ditandai dengan tanda pisah.

Tabel 4.3 Selisih Deskriptif Metrik Lighthouse (Vue.js–Svelte)

Metrik	Selisih mean	Perubahan terhadap Vue.js
FCP	-0,059	-0,02%
LCP	0,345	0,07%
Speed Index	3,334	1,11%
TBT	0,000	–
CLS	0,000000	0,00%

Seluruh observasi masih berada dalam kategori *Good* menurut batas pada Tabel 3.13. Nilai maksimum masing-masing framework ditunjukkan pada Tabel 4.4; kategori tersebut tidak menggantikan analisis selisih pasangan.

Tabel 4.4 Kategori Lighthouse Berdasarkan Nilai Maksimum Observasi

Metrik	Batas Good	Maks. Vue.js	Maks. Svelte	Kategori
FCP	$\leq 1\,800$ ms	304,521	299,355	Keduanya *Good*
LCP	$\leq 2\,500$ ms	516,394	516,862	Keduanya *Good*
Speed Index	$\leq 3\,400$ ms	338,075	311,989	Keduanya *Good*
TBT	≤ 200 ms	0,000	0,000	Keduanya *Good*
CLS	$\leq 0{,}1$	0,000587	0,000587	Keduanya *Good*

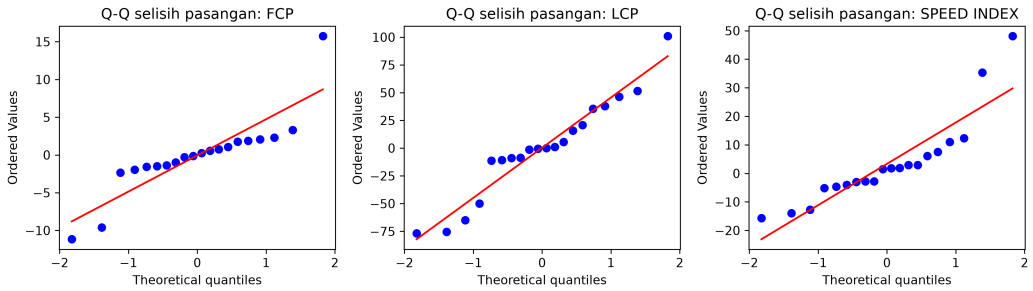
4.1.3 Statistika Inferensial Metrik Lighthouse

Unit inferensi adalah selisih dalam setiap blok, bukan dua kelompok run independen. Hipotesis untuk tiap metrik mengikuti Persamaan 3.5 pada Bab 3. Normalitas diperiksa pada 20 selisih pasangan menggunakan Shapiro–Wilk dan Q–Q plot. Tabel 4.5 menunjukkan nilai p; nilai untuk TBT dan CLS tidak dihitung karena semua selisih pasangan nol.

Tabel 4.5 Normalitas Shapiro–Wilk Selisih Pasangan

Metrik	n pasangan	p Shapiro–Wilk	Keputusan pada $\alpha = 0{,}05$
FCP	20	0,0012	Tidak normal
LCP	20	0,3268	Gagal menolak normalitas
Speed Index	20	0,0022	Tidak normal
TBT	20	–	Konstan
CLS	20	–	Konstan

Nilai p selisih FCP dan Speed Index di bawah 0,05 sehingga digunakan uji permutasi berpasangan dua arah. Selisih LCP tidak menunjukkan penolakan normalitas sehingga digunakan uji t berpasangan dua arah. Untuk TBT dan CLS, seluruh selisih bernilai nol dan notebook menggunakan kasus konstan pada uji permutasi. Gambar 4.2 memperlihatkan distribusi selisih untuk tiga metrik yang bervariasi.



Gambar 4.2 Q–Q Plot Selisih Pasangan FCP, LCP, dan Speed Index
Sumber: Diolah penulis dari 20 selisih pasangan Lighthouse.

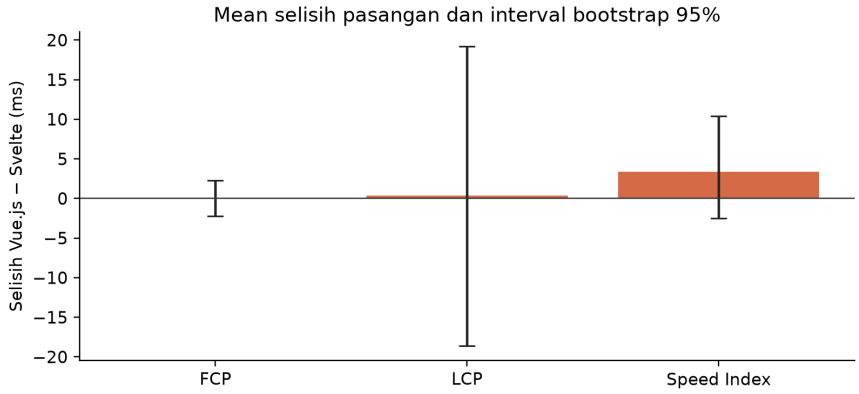
Uji permutasi membalik tanda selisih di dalam pasangan sebanyak 100.000 kali. Interval kepercayaan (CI) 95% mean selisih dihitung dengan *bootstrap percentile* 20.000 pengambilan ulang pasangan utuh. Ukuran efek yang dilaporkan

adalah Cohen d_z; nilai ini tidak terdefinisi ketika SD selisih nol. Lima nilai p mentah dikoreksi dengan prosedur Holm. Tabel 4.6 memuat hasil lengkap dalam presisi penyajian.

Tabel 4.6 Hasil Inferensial Berpasangan Metrik Lighthouse

Metrik	Mean d_i	CI 95%	Uji	p mentah	p Holm	Cohen d_z
FCP	-0,059	[-2,199; 2,260]	Permutasi	0,9683	1,0000	-0,011
LCP	0,345	[-18,583; 19,180]	t berpasangan	0,9725	1,0000	0,008
Speed Index	3,334	[-2,506; 10,415]	Permutasi	0,3714	1,0000	0,218
TBT	0,000	[0,000; 0,000]	Permutasi	1,0000	1,0000	–
CLS	0,000000	[0,000000; 0,000000]	Permutasi	1,0000	1,0000	–

Gambar 4.3 menampilkan mean selisih FCP, LCP, dan Speed Index bersama CI 95%. Ketiga interval melintasi nol. Setelah koreksi Holm, kelima nilai p bernilai 1,0000; tidak ada metrik yang menolak H_0 pada $\alpha = 0{,}05$. Hasil ini tidak membuktikan kedua framework identik, tetapi batch ini belum memberi bukti perbedaan mean pada kondisi yang diuji.



Gambar 4.3 Mean Selisih Pasangan dan CI Bootstrap 95%
Sumber: Diolah penulis dari keluaran notebook analisis.

4.1.4 Statistika Deskriptif Efisiensi Sumber Daya

Ukuran transfer JavaScript, total transfer, dan jumlah permintaan berasal dari audit `resource-summary`. Transfer JavaScript adalah `transferSize` resource bertipe `script`, bukan ukuran berkas sumber atau *bundle size* hasil build. Hasil deskriptifnya disajikan pada Tabel 4.7 dan Tabel 4.8.

Tabel 4.7 Statistika Deskriptif Efisiensi Sumber Daya: Pemusatan dan Penyebaran

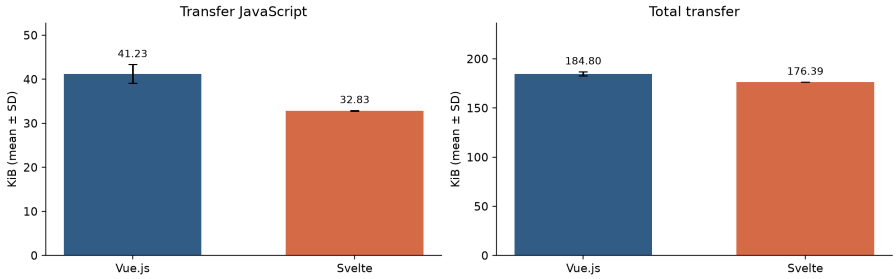
Metrik	Framework	n	Mean	SD	Median	IQR
Transfer JavaScript (byte)	Vue.js	20	42.222,45	2.177,34	42.694,00	2,25
Transfer JavaScript (byte)	Svelte	20	33.620,30	88,49	33.592,00	1,50
Total transfer (byte)	Vue.js	20	189.231,80	2.214,34	189.705,00	3,25
Total transfer (byte)	Svelte	20	180.625,50	103,11	180.603,00	3,00
Request count	Vue.js	20	12,90	0,45	13,00	0,00
Request count	Svelte	20	13,00	0,00	13,00	0,00

Tabel 4.8 Statistika Deskriptif Efisiensi Sumber Daya: Kuartil dan Rentang

Metrik	Framework	Q_1	Q_3	Minimum	Maksimum
Transfer JavaScript (byte)	Vue.js	42.692,75	42.695,00	32.976,00	42.982,00
Transfer JavaScript (byte)	Svelte	33.590,75	33.592,25	33.589,00	33.879,00
Total transfer (byte)	Vue.js	189.703,75	189.707,00	179.830,00	190.056,00
Total transfer (byte)	Svelte	180.601,00	180.604,00	180.444,00	180.917,00
Request count	Vue.js	13,00	13,00	11,00	13,00
Request count	Svelte	13,00	13,00	13,00	13,00

Mean transfer JavaScript Vue.js adalah 42.222,45 byte dan Svelte 33.620,30 byte; Svelte lebih rendah 8.602,15 byte atau 20,37% terhadap Vue.js. Mean total transfer Vue.js 189.231,80 byte dan Svelte 180.625,50 byte; selisihnya 8.606,30 byte atau 4,55%. Satu report Vue.js mencatat 11 permintaan sehingga mean Vue.js 12,90, sedangkan seluruh report Svelte mencatat 13 permintaan. Perbedaan jumlah permintaan ini hanya dilaporkan secara deskriptif.

Gambar 4.4 memperlihatkan perbandingan mean dan SD ukuran transfer. Jumlah permintaan disajikan pada tabel karena rentangnya jauh lebih kecil dan menggunakan satuan berbeda dari ukuran transfer.



Gambar 4.4 Perbandingan Mean dan SD Ukuran Transfer

Sumber: Diolah penulis dari keluaran notebook analisis.

Tabel 4.9 Selisih Deskriptif Efisiensi Sumber Daya (Vue.js–Svelte)

Metrik	Selisih mean	Perubahan terhadap Vue.js
Transfer JavaScript (byte)	8.602,15	20,37%
Total transfer (byte)	8.606,30	4,55%
Request count	-0,10	-0,78%

DAFTAR REFERENSI

Airbnb, Inc., 2026. *Q4 2025 Shareholder Letter*. 12 February. [online] https://www.sec.gov/Archives/edgar/data/1559720/000119312526048670/d58192dex991.htm. [Accessed 11 September 2026].

Azizullah, A., Rahayudi, B. and Priyambadha, B., 2026. Rancang Bangun Sistem Otomatisasi Pengujian Berbasis REST API Terintegrasi Dengan Playwright Pada PT XYZ. https://j-ptiik.ub.ac.id/index.php/j-ptiik/article/view/16105/7093

Cloudflare, 2026. SQL statements. *Cloudflare D1 Documentation*. [online] https://developers.cloudflare.com/d1/sql-api/sql-statements/. [Accessed 21 October 2026].

Deng, A., Du, M., Matlin, A. and Zhang, Q., 2023. Variance Reduction Using In-Experiment Data: Efficient and Targeted Online Measurement for Sparse and Delayed Outcomes. In: *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. https://doi.org/10.1145/3580305.3599928.

ElysiaJS, n.d. *Eden Treaty Overview*. [online] https://elysiajs.com/eden/treaty/overview.html#eden-treaty. [Accessed 11 September 2026].

Fahreza, N., Kharisma, A.P. and Marji, 2025. Analisis Perbandingan Performa Rendering Next.js dan SvelteKit pada Aplikasi E-commerce Berdasarkan Metrik Web Vitals. *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer*, 9(10). [online] https://j-ptiik.ub.ac.id/index.php/j-ptiik/article/view/15494/6870. [Accessed 28 August 2026].

Ferreira, F., Borges, H.S. and Valente, M.T., 2022. On the (un-)adoption of JavaScript front-end frameworks. *Software: Practice and Experience*, 52(4), pp.947–966. https://doi.org/10.1002/spe.3044.

Forsey, H., Leahy, D., Fields, B., Minocha, S., Attfield, S. and Snell, T., 2025. Designing for Learnability: Improvement Through Layered Interfaces. *Ergonomics in Design*, 33(3). https://doi.org/10.1177/10648046241273291.

Fusco, G., 2026. *Measuring User Experience with the UEQ: A Practical Guide and Case Study in the Hospitality Sector*. *Esario*. 17 June. [online] https://www.esario.com/blog/measuring-user-experience-with-the-ueq. [Accessed 17 September 2026].

Gaffney, K.P., Prammer, M., Brasfield, L., Hipp, D.R., Kennedy, D. and Patel, J.M., 2022. SQLite: Past, Present, and Future. *Proceedings of the VLDB Endowment*, 15(12), pp.3535–3547. https://doi.org/10.14778/3554821.3554842.

Google, n.d.a. Cumulative Layout Shift. *web.dev*. [online] https://web.dev/articles/cls. [Accessed 11 September 2026].

Google, n.d.b. First Contentful Paint. *Chrome for Developers*. [online] https://developer.chrome.com/docs/lighthouse/performance/first-contentful-paint. [Accessed 11 September 2026].

Google, n.d.c. Largest Contentful Paint. *web.dev*. [online] https://web.dev/articles/lcp. [Accessed 11 September 2026].

Google, n.d.d. Speed Index. *Chrome for Developers*. [online] https://developer.chrome.com/docs/lighthouse/performance/speed-index. [Accessed 11 September 2026].

Google, n.d.e. Total Blocking Time. *Chrome for Developers*. [online] https://developer.chrome.com/docs/lighthouse/performance/lighthouse-total-blocking-time. [Accessed 11 September 2026].

Guillermou, H., Abraham, C., Annesi-Maesano, I. and Molinari, N., 2024. Multiple hypothesis testing in allergy and hypersensitivity diseases investigation: a pedagogical perspective. *Journal of Allergy and Hypersensitivity Diseases*, 3–4, 100014. https://doi.org/10.1016/j.jahd.2024.100014.

Handasari, S.P., Wulandari, R. and Haikal, 2024. Evaluation of the Usability and User Experience of the Jaminan Kesehatan Nasional Mobile Application in Indonesia. *Healthcare Informatics Research*, 30(4), pp.324–332. https://doi.org/10.4258/hir.2024.30.4.324.

Holt, C.A. and Sullivan, S.P., 2023. Permutation tests for experimental data. *Experimental Economics*, 26(4), pp.775–812. https://doi.org/10.1007/s10683-023-09799-6.

ISO, 2018. *ISO 9241-11:2018 Ergonomics of human-system interaction—Part 11: Usability: Definitions and concepts*. [online] https://www.iso.org/standard/63500.html. [Accessed 11 September 2026].

ISO, 2019. *ISO 9241-210:2019 Ergonomics of human-system interaction—Part 210: Human-centred design for interactive systems*. [online] https://www.iso.org/standard/77520.html. [Accessed 11 September 2026].

Kushendriawan, M.A., Santoso, H.B., Putra, P.O.H. and Schrepp, M., 2021. Evaluating User Experience of a Mobile Health Application 'Halodoc' using User Experience Questionnaire and Usability Testing. *Jurnal Sistem Informasi*, 17(1), pp.58–71. https://doi.org/10.21609/jsi.v17i1.1063.

Microsoft, n.d. *Playwright Documentation*. [online] https://playwright.dev/docs/intro. [Accessed 11 September 2026].

Nguyen, M.H., Sedoc, J. and Taylor, C.O., 2024. Usability, Engagement, and Report Usefulness of Chatbot-Based Family Health History Data Collection: Mixed Methods Analysis. *Journal of Medical Internet Research*, 26, e55164. https://doi.org/10.2196/55164.

Mortazavi, E., Doyon-Poulin, P., Imbeau, D., Taraghi, M. and Robert, J.-M., 2024. Exploring the Landscape of UX Subjective Evaluation Tools and UX Dimensions: A Systematic Literature Review (2010–2021). *Interacting with Computers*, 36(4), pp.255–278. https://doi.org/10.1093/iwc/iwae017.

NIST/SEMATECH, n.d. *Anderson–Darling and Shapiro–Wilk Tests*. Engineering Statistics Handbook. [online] https://www.itl.nist.gov/div898/handbook/prc/section2/prc213.htm. [Accessed 25 September 2026].

Ollila, R., Mäkitalo, N. and Mikkonen, T., 2022. Modern Web Frameworks: A Comparison of Rendering Performance. *Journal of Web Engineering*. [online] https://doi.org/10.13052/jwe1540-9589.21311.

Phillips, M.R., Wykoff, C.C., Thabane, L., Bhandari, M. and Chaudhary, V., 2022. The clinician's guide to p values, confidence intervals, and magnitude of effects. *Eye*, 36, pp.341–342. https://doi.org/10.1038/s41433-021-01863-w.

Putra, F.P.E., Hasbullah, H., Muslim, F. and Paradina, R., 2025. Technical Performance Comparison of Modern Frontend Frameworks Study on Svelte, React, and Vue. *Brilliance: Research of Artificial Intelligence*, 5(1), pp.355–364. https://doi.org/10.47709/brilliance.v5i1.6133.

Schrepp, M., Hinderks, A. and Thomaschewski, J., 2017. Construction of a Benchmark for the User Experience Questionnaire (UEQ). *International Journal of Interactive Multimedia and Artificial Intelligence*, 4(4), pp.40–44. https://doi.org/10.9781/ijimai.2017.445.

Schrepp, M., 2023. *User Experience Questionnaire Handbook*, Version 11. [online] https://ueq-online.org/Material/Handbook.pdf. [Accessed 27 August 2026].

SciPy, n.d. *scipy.stats.shapiro*. SciPy Documentation. [online] https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.shapiro.html. [Accessed 25 September 2026].

SciPy, n.d. *scipy.stats.permutation_test*. SciPy Documentation. [online] https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.permutation_test.html. [Accessed 25 September 2026].

SciPy, n.d. *scipy.stats.ttest_rel*. SciPy Documentation. [online] https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ttest_rel.html. [Accessed 25 September 2026].

Shatz, I., 2024. Assumption-checking rather than (just) testing: The importance of visualization and effect size in statistical diagnostics. *Behavior Research Methods*, 56, pp.826–845. https://doi.org/10.3758/s13428-023-02072-x.

Sofi'ie, F.A.F. and Qoiriah, A., 2023. Analisis Perbandingan Framework Front-End Javascript React dan Vue Pada Pengembangan Website. *Journal of Informatics and Computer Science (JINACS)*, 5(02), pp.157–164. https://doi.org/10.26740/jinacs.v5n02.p157-164.

Svelte, n.d. *Overview*. Svelte Documentation. [online] https://svelte.dev/docs/svelte/overview. [Accessed 11 September 2026].

Vanicek, T. and Popelka, S., 2023. The Think-Aloud Method for Evaluating the Usability of a Regional Atlas. *ISPRS International Journal of Geo-Information*, 12(3), 95. https://doi.org/10.3390/ijgi12030095.

Wehner, N., Seufert, M., Schatz, R. and Hoßfeld, T., 2023. Do You Agree? Contrasting Google's Core Web Vitals and the Impact of Cookie Consent Banners with Actual Web QoE. *Quality and User Experience*, 8, 5. https://doi.org/10.1007/s41233-023-00058-3.

Vitest, n.d. *Vitest Guide*. [online] https://vitest.dev/guide/. [Accessed 11 September 2026].

Vue.js, n.d. *Rendering Mechanism*. Vue.js Documentation. [online] https://vuejs.org/guide/extras/rendering-mechanism. [Accessed 11 September 2026].